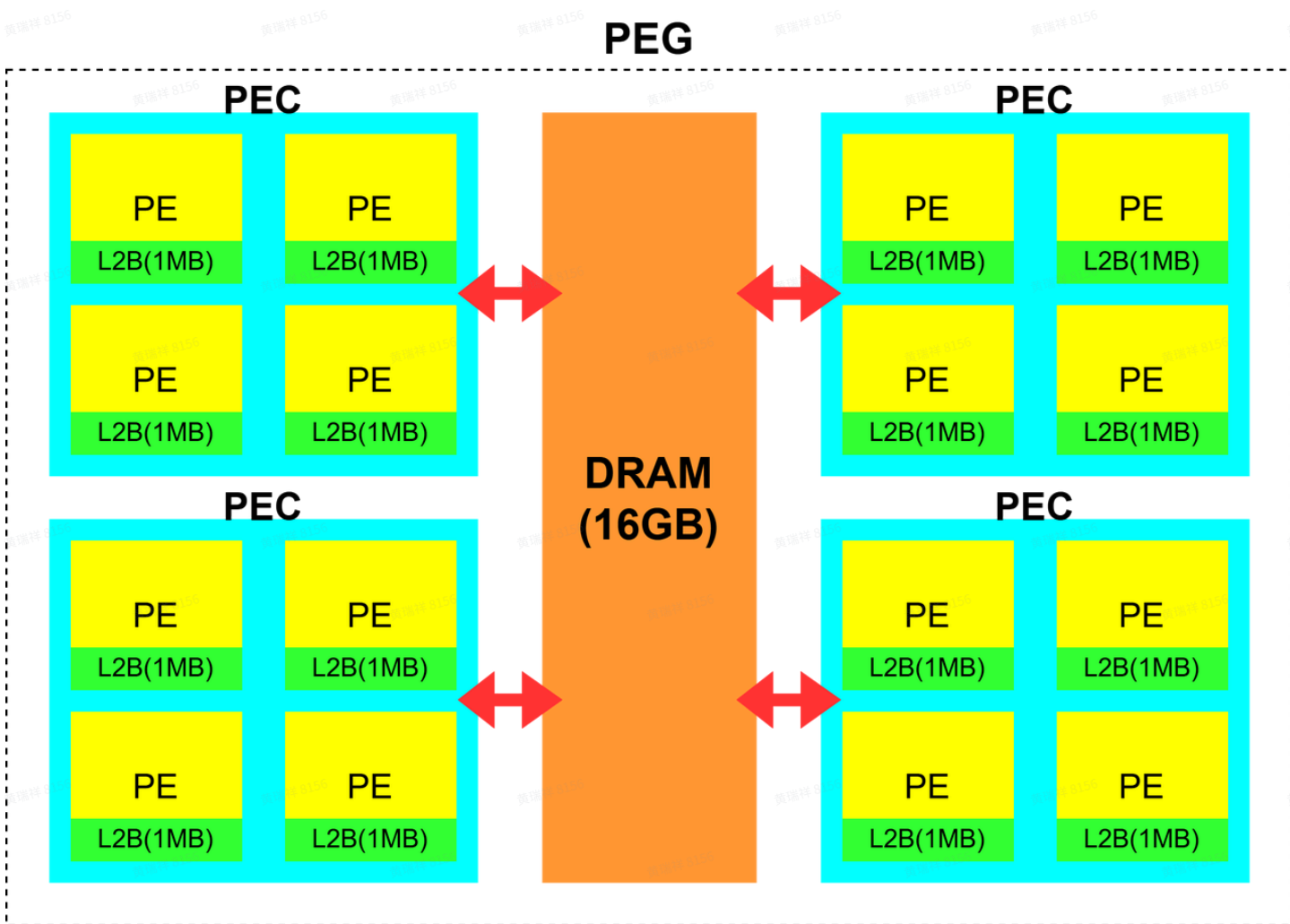


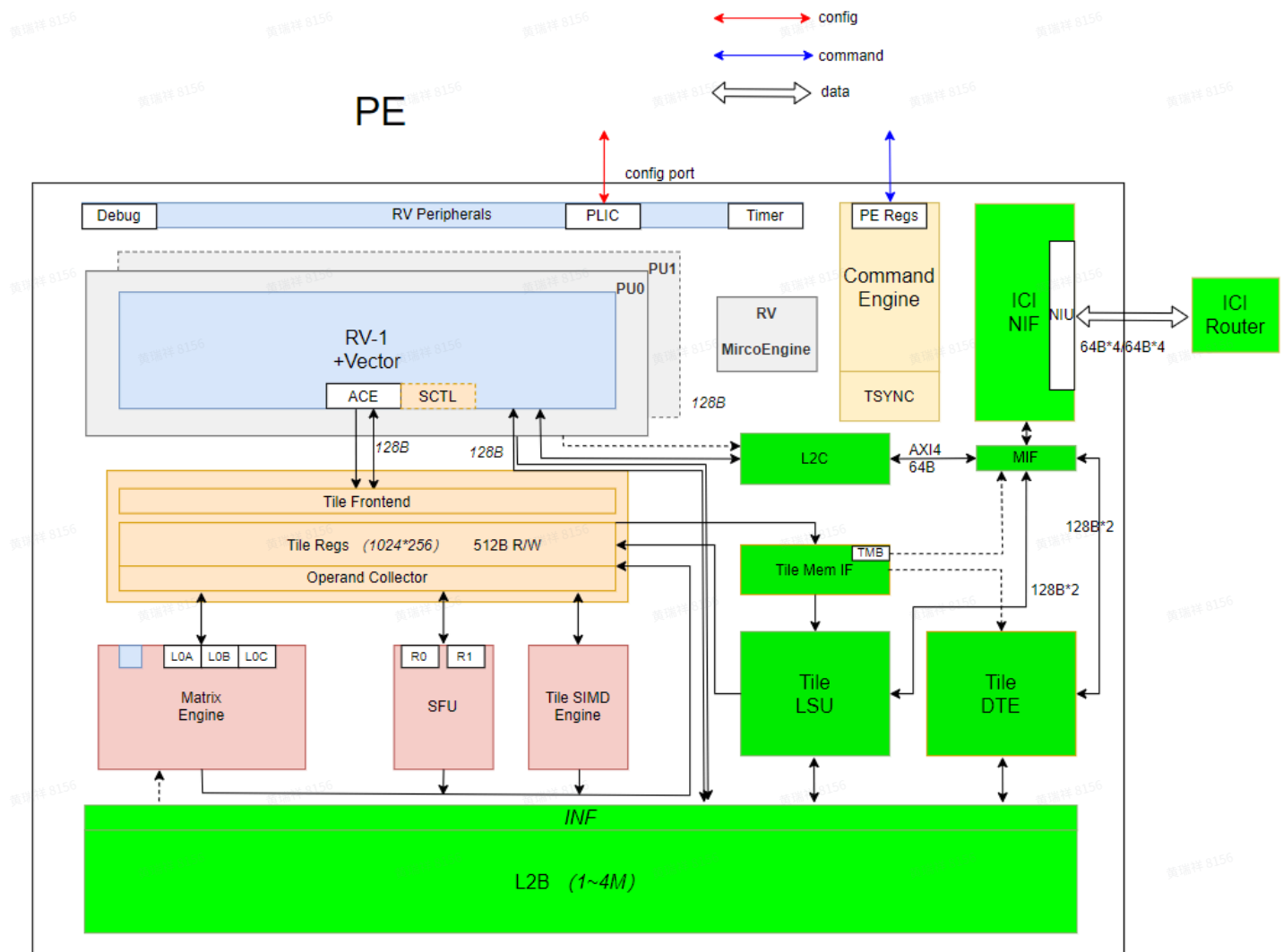
SIPU硬件架构总共包含2个PEG，从编程可用资源的角度出发，单个算子可自由调度使用的范畴为1个PEG（PEG内每个PEC对应的DRAM容量为4GB，4组PEC的DRAM interleave为一个整体，interleave的颗粒度为4KB），如下图所示：



其中：

- (1) PEG由4个PEC组成，每个PEC由4个基本的执行单元PE组成，每个PE由2个RV Core协作调度
- (2) 根据以上硬件组成情况，SIPU kernel各个维度可配置的情况为：
 - grid: 1-4096（大于物理限制，总线程数不能超过4096）
 - cluster: 1-4(逻辑上与物理上一致)
 - block: 1-2(逻辑上与物理上一致)
- (3) PEG内可使用的global memory大小为16GB（DRAM，如果包含ECC，可用大小会更小）
- (4) 每个PE内可使用的shared memory大小为1MB（L2B）

每个PE内部，又包含若干个硬件单元，如TLSU，TALU，DTE等等，如下图：



单个PE的基础参数有：

PE中Core个数：2个RV Core，1个Tile Core

RV Core scalar整型寄存器数量：32个reg（64 bit）

RV Core scalar浮点寄存器数量：32个reg（64 bit）

RV Core Vector寄存器数量：32个vec（1024 bit）

Tile Core Vector寄存器数量：256个TReg（1024 Byte）

L2B（shared memory）大小：1M Byte，2个RV Core + 1个Tile Core共享

L2C cache line：64 Byte

1.2 程序开发工具

SIPU算子开发，依赖运行时（runtime）提供的基础功能API，以及编译器（compiler）提供的Tile Core与RV Core API。目前的SDK依赖，位于公共开发环境（NAS）的以下目录：

代码块

```
1 /share_data/sipu_sdk_debug/
```

SDK包含了siformat, sipurt, SiTe, 以及交叉编译工具等一系列开发资源, 算子开发与编译执行, 需要首先source相关资源:

代码块

```
1 source
  /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/sipu_sdk_setup.sh
```

目前由runtime封装提供的编译工具为scc, 基础的使用方法为:

代码块

```
1 # 编译可执行文件
2 scc -o vectmul vectmul.su
3
4 # 编译动态链接库
5 scc --shared -o vectmul_lib.so vectmul_lib.su
```

除了直接使用编译工具进行编译, 更加推荐使用CMake进行集成, 示例如下:

代码块

```
1 cmake_minimum_required(VERSION 3.27)
2 project(MySipuApp)
3
4 find_package(scc REQUIRED)
5
6 scc_add_executable(v vectMul.su v1.su)
7 scc_target_include_directories(v ./include1 ./include2)
8 scc_target_compile_definitions(v ENABLE_DEVICE_KERNEL_LAUNCH=1)
9 scc_target_compile_options(v "-O0" "-Wall")
10 scc_install(v DESTINATION bin)
11
12 scc_add_library(v2 SHARED v2.cpp)
13 scc_target_compile_options(v2 "-O2")
14 scc_install(v2 DESTINATION lib)
```

集成的步骤说明如下:

1. 导入 scc:

```
find_package(scc REQUIRED)
```

2. 添加可执行文件：支持多个源文件（只能包含一个main函数），生成的可执行文件就是target (./v)
`scc_add_executable(v v1.su v2.su)`
3. 添加include path：支持多个路径
`scc_target_include_directories(v ./include1 /include2)`
4. 添加动态链接库：支持多个库，注意动态链接库只针对host侧，device没有动态链接功能（
`scc_target_link_libraries(v m json) // 为host侧程序添加libm 和 libjson`
5. 添加编译选项：
`scc_target_compile_options(v "-O0" "-Wall")`
6. 安装可执行文件：
`scc_install(v DESTINATION bin)`
7. 添加库文件：
`scc_add_library(v2 SHARED v2.su)`
8. 安装库文件：
`scc_install(v2 DESTINATION lib)`

1.3 编译器标识符

更多关于编译器提供的配置选项的内容，可参考：[SIPU Clang 命令行参数与属性索引](#)

1.3.1 基本属性

`__sipu_global__`，作用在函数上，表明该函数是一个 SIPU kernel

`__sipu_device__`，可作用在函数或全局变量上：

- (1) 作用在函数上时，表明该函数是一个 device 函数
- (2) 作用在全局变量上时，表明该变量存在于 device 侧

`__sipu_shared__`，作用在变量上，表明变量存储在 shared memory 空间

为简化使用，编译器将以上属性简化如下：

代码块

```
1  __global__
2  __device__
3  __shared__
```

如果某些定义内容（如typedef，define等）为device端专用内容，则需要使用宏（`__SIPU_ARCH__`）进行包裹，以免引起编译错误：

代码块

```
1  #ifdef __SIPU_ARCH__
```

```

2
3 typedef vint32m8_t fixed_vint32m8_t
  __attribute__((riscv_rvv_vector_bits(__riscv_v_fixed_vlen * 8)));
4
5 #endif

```

1.3.2 Tile寄存器分配

`treg_threading_mode`，作用在 SIPU kernel 上（即 `__global__` 函数），指定该 kernel 在不同线程间分配 Tile 寄存器的模式，可以接受的值包括：

代码块

- 1 `symmetric_multi_thread` (默认值)，每个线程最多分配 80 个寄存器
- 2 `asymmetric_multi_thread(experimental)`，两个线程非对称分配寄存器，thread 0 分配 160 个，thread 1 分配 0 个
- 3 `single_thread`，单线程模式，最多分配 160 个寄存器

关于 `asymmetric_multi_thread` 的具体用法，可参考 6.16 小节。

1.3.3 Tile指令保序

`disable_tile_scheduling`，可作用在 **global** 或 **device** 函数上。编译器对这些函数会尽量使汇编中的 Tile 指令使用源代码中的顺序。



注意：这个 attribute 会尽量保证代码块内指令的相对顺序，但并不会禁止编译器的所有优化，因此 tile 指令和相关代码不是完全不会被改变，例如

1. 包含 tile builtin 的 loop 仍然可以被 unroll
2. 如果 tile builtin 是一条纯计算指令 (如 `tadd`, `tmma`)，但结果没有被使用，仍然可能被编译器删除
3. loop 中的 tile 指令如果是一个不变量，编译器可能会把该指令移动到 loop 外

使用方法，可参考如下：

代码块

```

1 __attribute__((disable_tile_scheduling))
2 __global__ void foo() {}

```

相关JIRA: [S1SW-1654] 编译器提供函数级别的编译属性实现tile core的指令顺序与builtin的书写顺序一致 - Jira

2 SIPU中的线程组织

 需要注意，采用<<<>>>方法调用kernel时，内部的grid/block等参数，一定要配置为dim3类型，以便与可能存在的shared memory大小配置参数相区分：

(1) 正确用法

```
dim3 grid{1};  
dim3 block{2};  
test<<<grid, block>>>();
```

(2) 错误用法

```
test<<<4, 1, 2>>>();
```

2.1 SIPU中的Hello World程序

SIPU代码与CUDA代码类似，也包含主机（host）端代码，和设备（device）端代码，一个简单的Hello World代码如下所示：

代码块

```
1  #include <sipu_runtime.h>  
2  #include <cstdio>  
3  
4  __global__ void hello_from_sipu(void) {  
5      printf("Hello World from SIPU\n");  
6  };  
7  
8  int main(int argc, char* argv[]) {  
9      dim3 grid{1};  
10     dim3 block{1};  
11  
12     hello_from_sipu<<<grid, block>>>();  
13     sipuDeviceSynchronize();  
14  
15     return 0;  
16 }
```

程序编译执行后，会有如下输出：

代码块

```
1  # 编译
2  scc -o hello hello.su
3
4  # 运行
5  ./hello
6
7  # 输出
8  [sipu-core-000]: Hello World from SIPU
```

2.2 SIPU中的线程组织

算子的核函数（kernel）中，可以指派多个线程，即通过配置grid，cluster，block来调度配置SIPU的硬件资源。结合SIPU硬件简介中的内容，cluster与block，对应的是物理上的硬件资源，因此有：

代码块

```
1  grid合法配置参数区间为：1-4096（总线程数不能超过4096）
2  cluster合法配置参数区间为：1-4，仅用于cooperative launch模式，实现跨PE的shared memory访问，参考6.18节（或者在普通模式下，设置为1）
3  block合法配置参数区间为：1-2
```

grid对应的是逻辑数量，不受硬件资源数量的限制，可以根据需要自行配置，或者将grid参数固定为16，由kernel内部进行任务的循环迭代。举例：blockDim为2，则gridDim不能大于 $4096/2=2048$ 。

一个启动了PEG中所有线程的代码示例如下：

代码块

```
1  #include <sipu_runtime.h>
2  #include <stdio>
3
4  __global__ void hello_from_sipu(void) {
5      printf("Hello World from SIPU\n");
6  };
7
8  int main(int argc, char* argv[]) {
9      dim3 grid{16};
10     dim3 block{2};
11
12     hello_from_sipu<<<grid, block>>>();
```



```
13     sipuDeviceSynchronize();
14
15     return 0;
16 }
```

输出信息如下：

代码块

```
1  [sipu-core-001]: Hello World from SIPU
2  [sipu-core-003]: Hello World from SIPU
3  [sipu-core-005]: Hello World from SIPU
4  [sipu-core-007]: Hello World from SIPU
5  [sipu-core-009]: Hello World from SIPU
6  [sipu-core-011]: Hello World from SIPU
7  [sipu-core-013]: Hello World from SIPU
8  [sipu-core-015]: Hello World from SIPU
9  [sipu-core-017]: Hello World from SIPU
10 [sipu-core-019]: Hello World from SIPU
11 [sipu-core-021]: Hello World from SIPU
12 [sipu-core-023]: Hello World from SIPU
13 [sipu-core-025]: Hello World from SIPU
14 [sipu-core-027]: Hello World from SIPU
15 [sipu-core-029]: Hello World from SIPU
16 [sipu-core-031]: Hello World from SIPU
17 [sipu-core-000]: Hello World from SIPU
18 [sipu-core-002]: Hello World from SIPU
19 [sipu-core-004]: Hello World from SIPU
20 [sipu-core-006]: Hello World from SIPU
21 [sipu-core-008]: Hello World from SIPU
22 [sipu-core-010]: Hello World from SIPU
23 [sipu-core-012]: Hello World from SIPU
24 [sipu-core-014]: Hello World from SIPU
25 [sipu-core-016]: Hello World from SIPU
26 [sipu-core-018]: Hello World from SIPU
27 [sipu-core-020]: Hello World from SIPU
28 [sipu-core-022]: Hello World from SIPU
29 [sipu-core-024]: Hello World from SIPU
30 [sipu-core-026]: Hello World from SIPU
31 [sipu-core-028]: Hello World from SIPU
32 [sipu-core-030]: Hello World from SIPU
```

可见32个线程随机无序的执行，分别输出了带有自己core标识的Hello World信息。

3 简单SIPU程序的基本框架

3.1 数组相加

我们以数组相加为例，介绍SIPU程序的基本框架，代码示例如下：

代码块

```
1  #include <sipu_runtime.h>
2
3  __global__ void add(const int* A, const int* B, int* C, int n) {
4      uint32_t id = blockDim.x * blockIdx.x + threadIdx.x;
5      uint32_t step = blockDim.x * gridDim.x;
6
7      for(uint32_t i = id; i < n; i+=step) {
8          C[i] = A[i] + B[i];
9      }
10 };
11
12 #define N 1024
13 int main(int argc, char* argv[]) {
14     size_t dsize = N * sizeof(int);
15     int host_A[N];
16     int host_B[N];
17     int host_C[N];
18
19     int* data_a = nullptr;
20     int* data_b = nullptr;
21     int* data_c = nullptr;
22     sipuMalloc(&data_a, dsize);
23     sipuMalloc(&data_b, dsize);
24     sipuMalloc(&data_c, dsize);
25
26     sipuMemcpy(data_a, host_A, dsize, sipuMemcpyHostToDevice);
27     sipuMemcpy(data_b, host_B, dsize, sipuMemcpyHostToDevice);
28
29     dim3 grid{16};
30     dim3 block{2};
31     add<<<grid, block>>>(data_a, data_b, data_c, N);
32
33     sipuMemcpy(host_C, data_c, dsize, sipuMemcpyDeviceToHost);
34
35     sipuFree(data_a);
36     sipuFree(data_b);
37     sipuFree(data_c);
38
39     return 0;
```

```
40 }  
41
```

3.2 程序框架

一个典型的SIPU算子的代码逻辑，如下：

代码块

```
1  1. 隐形的设备初始化  
2      在SIPU runtime API中，没有明显地初始化设备的函数。在第一次调用一个runtime API函数  
   时，设备将自动地初始化。  
3  
4  2. 设备内存的分配与释放  
5  sipuError_t sipuMalloc(T **devPtr, size_t size);  
6  sipuError_t sipuFree(void *devPtr);  
7  
8  3. HOST与DEVICE之间的数据传递  
9  sipuError_t sipuMemcpy(void *dst, const void *src, size_t count, enum  
   sipuMemcpyKind kind);  
10  数据传递的类型有：  
11  /**  
12   * SIPU memory copy types  
13   */  
14  enum __device_builtin__ sipuMemcpyKind  
15  {  
16      sipuMemcpyHostToHost      = 0,      /**< Host -> Host */  
17      sipuMemcpyHostToDevice    = 1,      /**< Host -> Device */  
18      sipuMemcpyDeviceToHost    = 2,      /**< Device -> Host */  
19      sipuMemcpyDeviceToDevice  = 3,      /**< Device -> Device */  
20      sipuMemcpyDefault         = 4        /**< Direction of the transfer  
   is inferred from the pointer values. Requires unified virtual addressing */  
21  };  
22  
23  4. 核函数中数据与线程的对应  
24  uint32_t id = blockDim.x * blockIdx.x + threadIdx.x;  
25  示例的加法程序中，每个线程根据自己的线程号去索引数据，每次循环迭代增加总的线程数去索引自己  
   线程的下一个数据，这种计算逻辑，能够将基础的运算执行逻辑，匹配到多个block与多个cluster的  
   并发使用上。
```

4 SIPU程序的错误检测

用于检测SIPU运行时错误的宏函数实例，如下：

代码块

```
1  #include <stdio.h>
2
3  #define SIPU_CHECK(call) \
4  do { \
5      sipuError_t err = call; \
6      if (err != sipuSuccess) { \
7          printf("SIPU Error at %s:%d - %s\n", __FILE__, __LINE__,
8              sipuGetErrorString(err)); \
9          exit(EXIT_FAILURE); \
10     } \
11 } while (0)
```

示例：

代码块

```
1  SIPU_CHECK(sipuMemcpy(data_a, host_A, dsize, sipuMemcpyHostToDevice));
```

5 常用指令

目前kernel开发主要使用的指令为Tile Core builtin指令集合，与RV Core builtin指令集合，可参考如下：

5.1 Tile Core常用指令

[☞ SIPU Tile Core常用指令归纳](#)

5.2 RV Core常用指令

[☞ SIPU RV Core常用指令归纳](#)

5.3 常用builtin快查表

[☞ 常用builtin快查表](#)

6 开发特性

6.1 平衡寄存器使用与循环依赖

由于PE中访存和计算使用不同的硬件单元，如果循环中同时包含访存与计算，那么可以通过减少访存与计算的相关依赖，来提高指令流水效率，示例如下：

代码块

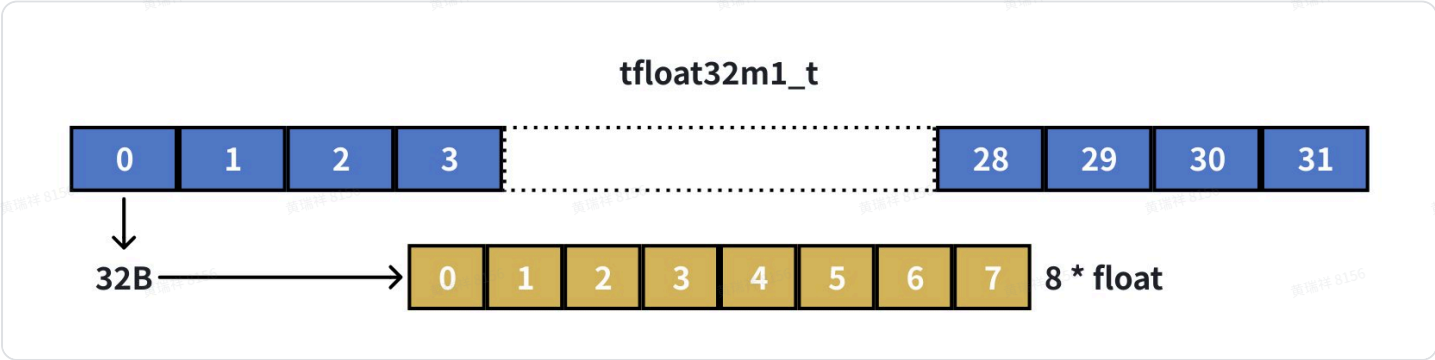
```
1 // 原始代码
2 vfloat32m1_t v1;
3 vfloat32m1_t v2 = __riscv_vfmv_v_f_f32m1(0.0f, vl);
4 for (int i = 0; i < 8; i++) {
5     v1 = tmv_vtr_f32(sum_vec, i);
6     v2 = __riscv_vfredosum_vs_f32m1_f32m1(v1, v2, vl);
7 }
8 float sum = __riscv_vfmv_f_s_f32m1_f32(v2);
```

以上代码在多次循环中，使用相同的register v1，限制了访存与计算的流水并发能力，可以调整如下：

代码块

```
1 // 优化方式
2 typedef vfloat32m1_t fixed_vfloat32m1_t
   __attribute__((riscv_rvv_vector_bits(__riscv_v_fixed_vlen * 1)));
3 fixed_vfloat32m1_t vvec[9];
4 vvec[8] = __riscv_vfmv_v_f_f32m1(0.0f, vl);
5 for (int i = 0; i < 8; i++) {
6     vvec[i] = tmv_vtr_f32(sum_vec, i);
7     vvec[8] = __riscv_vfredosum_vs_f32m1_f32m1(vvec[i], vvec[8], vl);
8 }
9 float sum = __riscv_vfmv_f_s_f32m1_f32(vvec[8]);
```

6.2 循环中的MV指令使用须知



代码块

```

1 // 该接口只能修改1个Tile (1024B) 中的32B对齐的op1参数指定的32bit内容（如1个f32或2个f16）
2 // op1的合法取值范围为[0, 31]，0对应头部，31对应尾部
3 // 如果希望为Tile赋值（初始化）时，多次循环调用，只会有最后一次生效
4 tfloat32m1_t tmv_t_f32(float op0, unsigned long op1);
5 tfloat16m1_t tmv_t_f16(float16_t op0, unsigned long op1);
6 tbfloat16m1_t tmv_t_bf16(bfloat16_t op0, unsigned long op1);
7
8 // 该接口使用RVV对Tile Reg进行赋值，每次调用会更新Tile Reg中一个RV Vector大小的空间
9 // op1的合法取值范围为[0, 7]
10 // 如果希望为Tile赋值（初始化）时，多次循环调用，只会有最后一次生效
11 tfloat32m1_t tmv_t_f32(vfloat32m1_t op0, unsigned long op1);
12 tfloat16m1_t tmv_t_f16(vfloat16m1_t op0, unsigned long op1);
13 tbfloat16m1_t tmv_t_bf16(vbfloat16m1_t op0, unsigned long op1);
14
15 // 该接口可以服务于Tile的赋值（初始化），支持多次循环使用
16 tfloat32m1_t tmv_t_f32(tfloat32m1_t op0, float op1, unsigned long op2);
17 tfloat16m1_t tmv_t_f16(tfloat16m1_t op0, float16_t op1, unsigned long op2);
18 tbfloat16m1_t tmv_t_bf16(tbfloat16m1_t op0, bfloat16_t op1, unsigned long op2);
19
20 // 该接口可用于整个Tile初始化数值相同的情况
21 tfloat32m1_t tmv_t_f32(float op0);
22 tfloat16m1_t tmv_t_f16(float16_t op0);
23 tbfloat16m1_t tmv_t_bf16(bfloat16_t op0);

```

6.3 API报错机制

由于目前开发实现的算子API大部分为无返回值类型，算子封装与实现逻辑中，需要有更显著的提示报错方式，可利用现有check接口进行报错输出，检测类型一般可包括：

- (1) kernel调用出错
- (2) 数据类型不支持
- (3) 其他调用参数未符合预设条件

等等。

检测方式可参考如下：

代码块

```
1 sipu::check(检测条件, "检测输出", __FILE__, __LINE__);
```

示例如下：

代码块

```
1 sipu::check(ret == SIPU_SUCCESS, "SiLaunchKernel error", __FILE__, __LINE__);
```

6.4 Tile与RV Vector寄存器调试输出

由于Tile Core与RV Core均是按块访存与运算，可能存在需要评估单个元素计算结果的情况，如：

代码块

```
1 // 该语句为tvec赋初值，如果需要检查所有元素是否正确赋0，则没有对应的print接口
2 tfloat32m1_t tvec = tmv_tir_broadcast_f32(0, 0);
3
4 // 由于Tile register (RV Vector register) 也是一段连续存储空间，可以通过强制类型转换获得首地址，然后进行循环或定位输出
5 float *tmp = (float *)&tvec;
6 for (int i = 0; i < 256; i++) {
7     printf("tvec[%d] = %f\n", tmp[i]);
8 }
```

6.5 巧用UNION (tfloat32m2)

由于当前Tile Core指令还不支持M2级别的基础乘法（以及其他ALU和SFU运算），需要一些work around操作来实现相关功能，此时就需要灵活使用union类型，如下（runtime已经声明了相关union类型，可直接使用，以下以M2级别的float为例）：

代码块

```
1 typedef union
2 {
3     struct
4     {
5         tfloat32m1_t m1e0;
6         tfloat32m1_t m1e1;
7     };
8     tfloat32m2_t m2e0;
9 } tfloat32m2;
```

数据访存是支持M2级别的操作的，所以可以形成类似tfloat32m2_t的数据对象，但是这种数据对象暂时还没有对应的Tile Core ALU指令，所以可以按照union的方式进行拆解实现，如下：

代码块

```
1 tfloat32m2 v1, v2, v3;
2 v1.m2e0 = tld_linear_global_m2(input1, 0, 0);
3 v2.m2e0 = tld_linear_global_m2(input2, 0, 0);
```

```
4 v3.m1e0 = tmul(v1.m1e0, v2.m1e0);
5 v3.m1e1 = tmul(v1.m1e1, v2.m1e1);
6 tst_linear_global_m2(v3, output, 0, 0);
```

6.6 调试输出 (debug print)

kernel开发过程中，经常需要打印输出数值信息，会使用到printf函数，但是release版本是不允许打印输出信息存在的。为了避免开发过程中，忘记注释/删除printf语句，调试输出，可统一使用以下指令（用法与printf类似）：

代码块

```
1 #include "sipu_kernel_debug.h"
2
3 DEBUG_PRINT("test output: A = %d, B = %f\n", A, B);
```

代码调试时，可通过增加编译参数，打开调试输出功能：

代码块

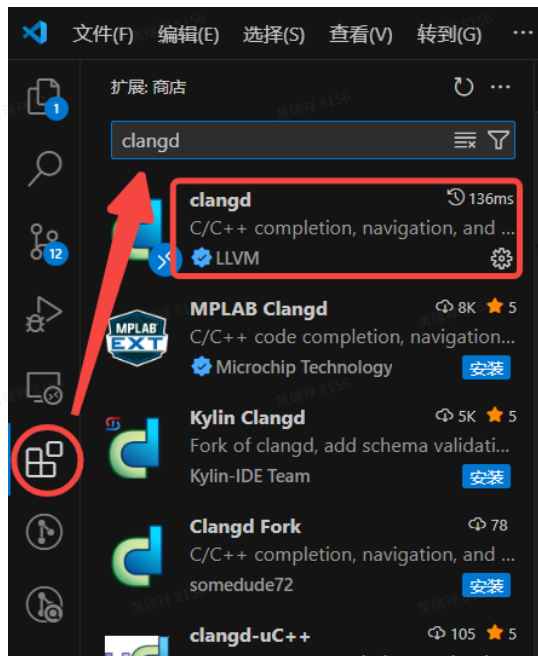
```
1 --clangopt="-DKERNEL_DBG"
```

代码提交时，需要将编译参数移除。

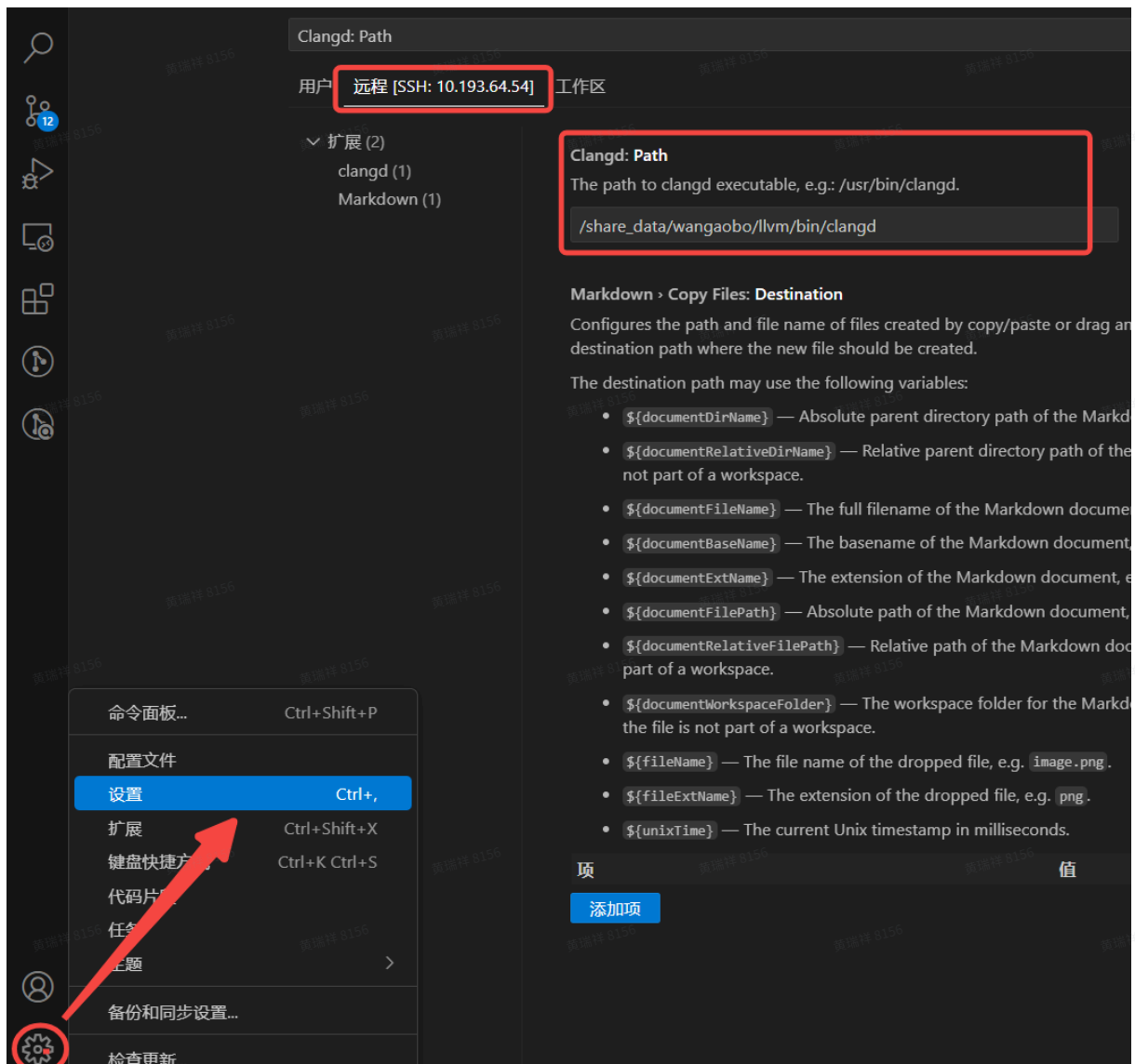
6.7 clangd插件使用

VSCode可以通过安装配置clangd插件，自动提示可用的Tile Core指令，配置方法如下：

(1) VSCode扩展中，搜索安装clangd插件



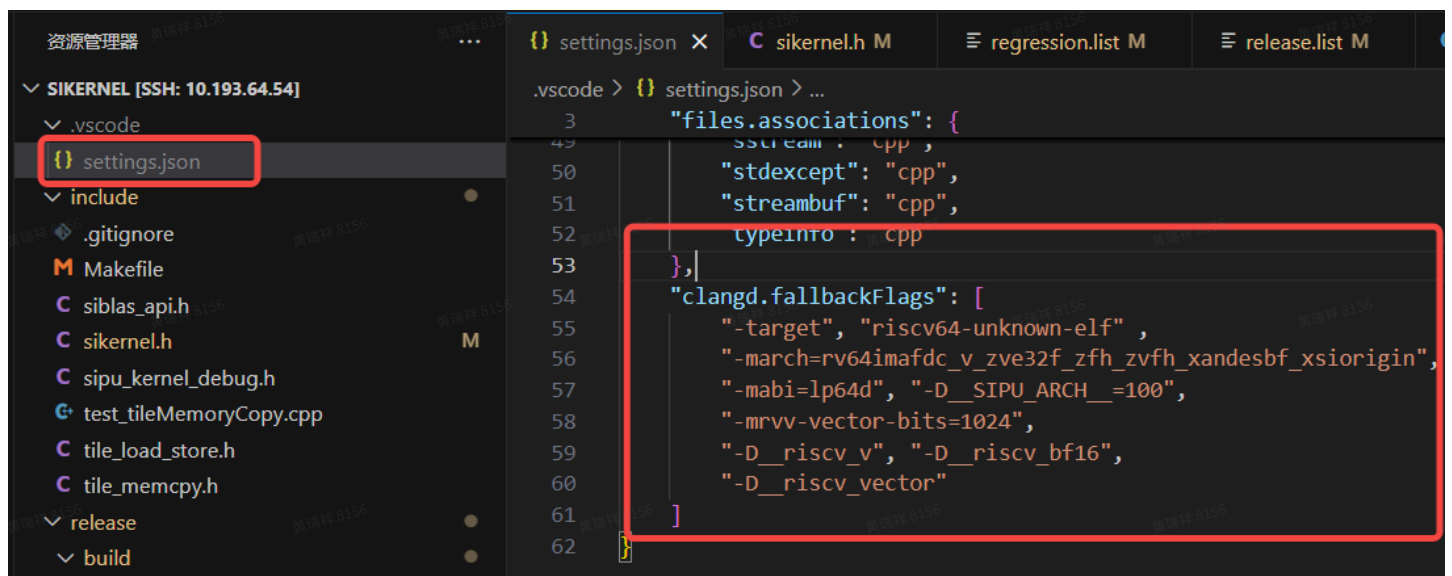
(2) VSCode设置中，搜索“Clangd: Path”，修改内容为：
“/share_data/wangaobo/llvm/bin/clangd”



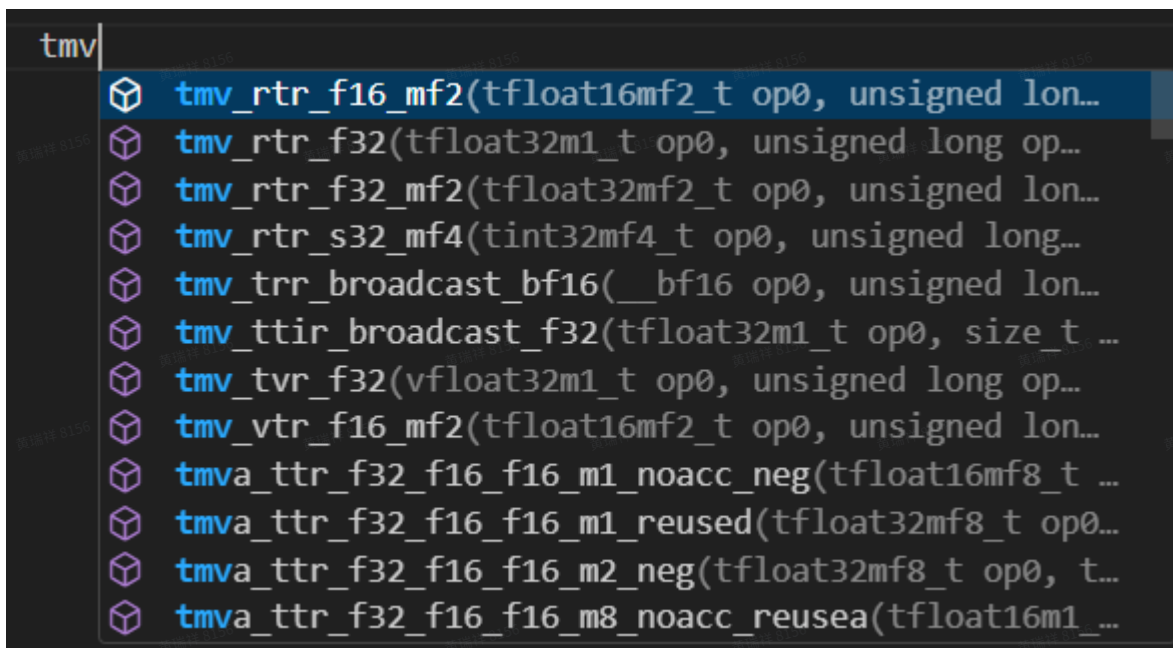
(3) 代码项目的根目录下，.vscode目录内，修改settings.json文件，增加如下内容（注意新增字段与上个字段，需要用逗号分隔）：

代码块

```
1  "clangd.fallbackFlags": [  
2      "-target", "riscv64-unknown-elf" ,  
3      "-march=rv64imafdc_v_zve32f_zfh_zvf_h_xandesbf_xsiorigin",  
4      "-mabi=lp64d", "-D__SIPU_ARCH__=100",  
5      "-mrvv-vector-bits=1024",  
6      "-D__riscv_v", "-D__riscv_bf16",  
7      "-D__riscv_vector"  
8  ]
```



配置完成后，代码中输入RV Core指令或Tile Core指令前缀时，会自动提示可能存在的完整指令名称，便于选用开发，效果如下图所示：



6.8 RV Core cache line规避

3.1.3 Risc-V Core Memory Model

3.1.3.1 RVWMO

AX45MPV采用RVWMO的Memory Model，详情可见 RVWMO Memory Consistency Model ([riscv-unprivileged.pdf](#)中的章节)

简单来说，所有同地址访问和重叠的地址区域中按照Program Order执行。

寄存器读写按照Program Order执行 (和Tile Core有区别)

3.1.3.2 Cluster间Coherency以64B Cacheline为粒度

多个PE间 (即多个CPU Cluster之间) 不允许在未进行同步的情况下，使用RV读写同一个64B Cacheline之内地址。(此限制为Andes AX45MPV的独有限制)

举例：

PE_0的RV写了0x40-0x48的地址，PE_1的RV写了0x60-0x68的地址。PE_0 先进行Cache Flush操作写回到Dram，PE_1在此之后进行Cache Flush写回到Dram，这种情况下，只有PE_1写的0x60-0x68写入到Dram去，PE_0的写行为失效了。

可行方案：

- 每个PE仅使用某个64B，软件感知这种限制
- 同时操作同一个64B时，将这部分地址空间设置为non-cacheable

算子开发过程中，尽量减少RV Core与global memory的交互。(该问题仍在讨论中，目前没有确定解决方案)

另外，由于kernel内部参数在运行时可能存在于栈空间，RV Core与Tile Core互访仍然存在cache line冲突问题，所以要求Tile Core不要**主动**访问kernel代码内的栈空间 (Tile Reg寄存器溢出至栈空间等被动情况除外)。

6.9 Kernel Function调用层级

算子的kernel代码中，函数调用不要超过4层 (RAS只有4级)，只调用少数几次的函数或者较短的函数建议都使用inline标记。RAS预测器在call执行4次后会将新的top覆盖最老的“call指令+4”地址，会产生预测错误的结果影响性能。

6.10 分支语句使用

尽量使用if else语句，不要或少使用switch case语句。因为没有间接跳转预测器，JALR类根据src reg的跳转是在现有分支预测架构下准确度很差。if else语句尽量用于判断thread_id等，这样每次预测的结果都是同一个结果，易于让分支预测器预测。

6.11 循环语句使用

循环体（如for和while）和函数内，建议将标量指令密集摆放在靠前位置。RVV/ACE/Tile Core和CSR操作指令是不能推测执行的。位置无关的标量指令建议更多放在代码段前面的位置。

每个循环尽量大，减少分支预测产生的bubble。AX45MPV分支预测器需要2个cycle才能产生预测结果，F1 cycle从ICache Cacheline取指的结果会被废弃。因此，尽量将循环扩大，且循环内尽量不要用到跳转指令。这样大概率这些bubble可以被指令执行时间掩盖掉。另外，分支预测错误的情况会产生5 cycles的bubble（直接跳转），7 cycles（无条件间接跳转）

TALU/TSFU/TMAC 最好不用小循环：

代码块

```
1  0x20: tadd.ttt T12, T4, T0
2  0x28: beq x7, x8, 0x20
```

tadd分支预测7个cycle，但是tadd产生的寄存器依赖需要14个cycle，所以小循环内tile core指令性能会比较差。

6.12 寄存器MOVE方法

整个寄存器MOVE时，建议使用tadd 0。由于ACE发射效率受限为1 inst/cycle，tadd指令做tile reg move，使用的指令数更少，后续指令发射效率会提高。Tile move和CSR读写代码如下：

代码块

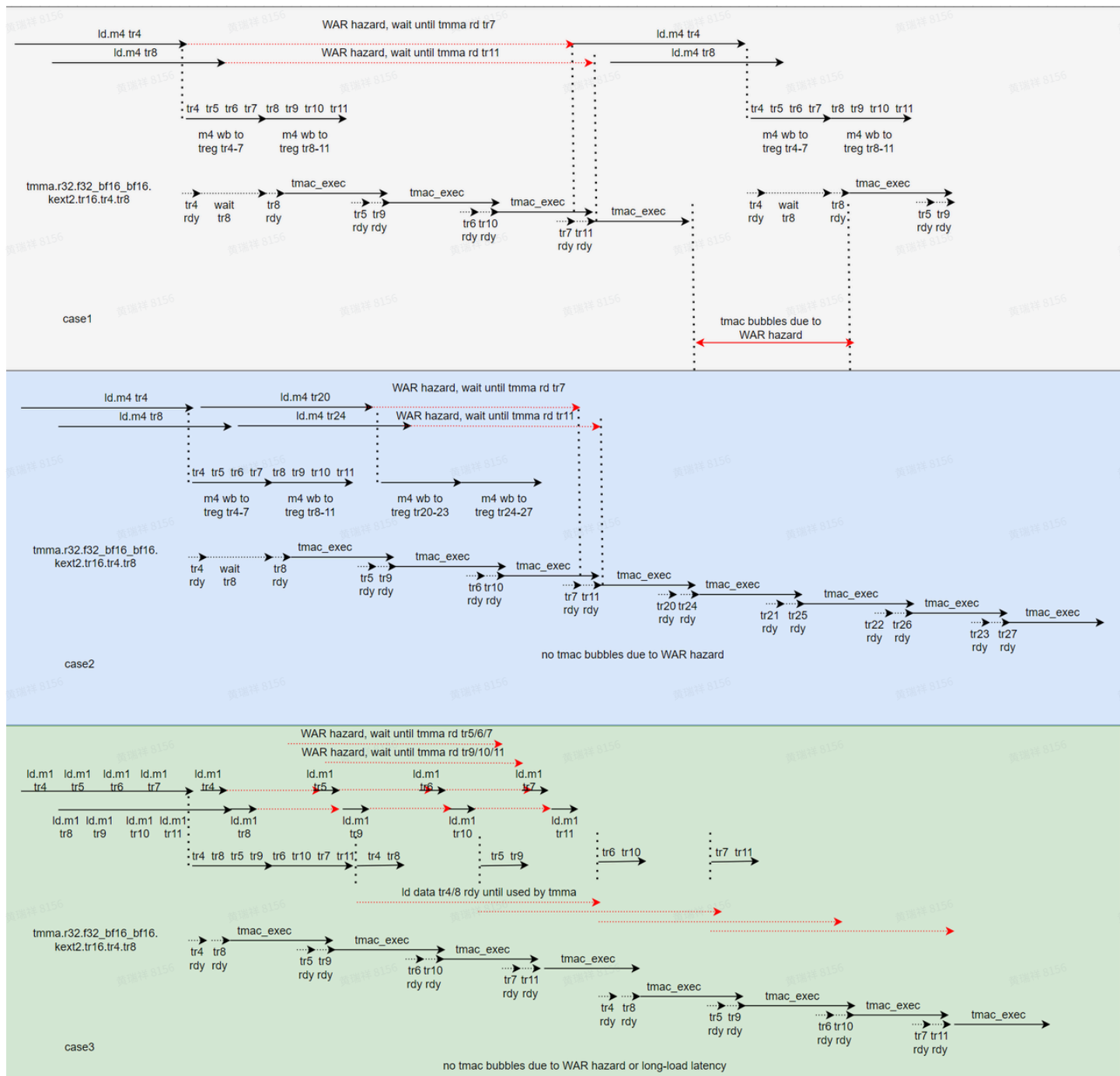
```
1  0x20: tmv.ttrr.u32 T12, T4, x1, x2
2  0x28: beq x7, x8, 0x20
```

tmv指令：2个cycle发射，1个cycle执行。第一次分支预测错误，需要5 cycles才能拿到得知branch taken，下一个tmv指令需要8 cycles后才能发射。之后的分支预测，即使预测正确，由于Tile Core指令不可推测执行（speulative Execution）每次仍得花费8 Cycles才能执行要给tmv指令。所以在这种循环中，开销主要是不能推测执行。

两个寄存器间的tmov指令，最好用tadd指令，指令发射不会成为瓶颈。

6.13 LSU/TMAC指令配比

算子kernel代码需要注意load/store/mma指令的配比，某一类指令不能太多，一类指令太多可能导致指令发不下去。load/store建议使用M1，硬件解依赖M1最快，如下图三个case流程图分析：



6.14 首地址对齐

TLSU单元，与TDTE单元，对于数据访存的首地址，是有对齐要求的，具体如下：

	指令示例	首地址对齐限制
TLSU	tld_linear_share_m1	Linear Format: 32B
	tld_linear_global_m2	Tiled Format: 256B
	tld_blk_global_m1	
TDTE	tacp_linear	Linear Format: 64B

tacp_tile_mbar_dsttm	Tiled Format: 256B
----------------------	--------------------

对齐要求，对于shared memory（L2B），global memory（DRAM）都适用；对于src addr，与dst addr，也均适用。

由于存在此限制，runtime的sipuMalloc接口，申请得到的global memory，首地址默认是256B对齐的。但是如果对于一个整块排列的tensor数据，进行slice操作，某些情况下得到的新的tensor data的首地址，可能存在非对齐的情况，需要谨慎对待。

6.15 正负max（接近无穷但不是无穷）的使用

arch定义的special number如下：

imm_encode	special_number	F32	F16	BF16
0	0	0x00000000	0x0000	0x0000
1	0.5	0x3f000000	0x3800	0x3f00
2	-0.5	0xbf000000	0xb800	0xbf00
3	1.0	0x3f800000	0x3c00	0x3f80
4	-1.0	0xbf800000	0xbc00	0xbf80
5	2.0	0x40000000	0x4000	0x4000
6	-2.0	0xc0000000	0xc000	0xc000
7	4.0	0x40800000	0x4400	0x4080
8	-4.0	0xc0800000	0xc400	0xc080
9	1.0/(2*PI)	0x3e22f983	0x3118	0x3e23
10	log2(e)	0x3fb8aa3b	0x3dc5	0x3fb9
11	NaN	0x7fc00000	0x7e00	0x7fc0
12	Max	0x7f7fffff	0x7bff	0x7f7f
13	Min	0xff7fffff	0xfbff	0xff7f
14	1e-5	3727c5ac	00a7	3728
15	1e-6	358637bd	0010	3586

当我们需要在kernel中给treg赋值为负无穷时，我们需要使用index为13的这个special number，但是在host侧，如果使用C语言的-`INFINITY`，则会发现有些计算结果host和kernel对不上。

这是因为，host侧的-`INFINITY`是0xff800000,是真正的负无穷，但这个值参与普通计算是有限制的，而且与我们kernel侧arch定义的0xff7fffff不同，所以我们需要在host侧同样使用0xff7fffff。而C语言中-`FLT_MAX`就是这个值，因此我们可以使用这个-`FLT_MAX`来赋值。

但是更为通用的方法是可以设置任意的十六进制，比如我们有时候想使用arch定义的其他special number，这个时候我们在host侧就可以这么定义，如下所示：

代码块

```
1  #define FLOAT_FROM_HEX(hexval) \
2      ({ \
3          float _f; \
4          uint32_t _bits = (uint32_t)(hexval); \
5          memcpy(&_f, &_bits, sizeof(_f)); \
6          _f; \
7      })
8
9  int main() {
10     float f1 = FLOAT_FROM_HEX(0xFF7FFFFFFF); // -FLT_MAX
11     float f2 = FLOAT_FROM_HEX(0xFF800000); // -INFINITY
12
13     printf("f1 = %e (0xFF7FFFFFFF)\n", f1);
14     printf("f2 = %e (0xFF800000)\n", f2);
15     return 0;
16 }
```

6.16 非对称TREG使用



双线程使用160个TREG的限定条件：

1. block保持双线程配置
2. TREG实际处于共享状态，需要开发者保证160（<=160均可）个TREG仅由某个线程全部使用，另一个线程TREG用量为0
3. 需要注意此模式下的同步逻辑，某些情况下可能需要主动调用tb sync指令，来保证同步

采用kernel function attribute进行区分，attribute类型为treg_threading_mode，支持3种模式类型，如下：

- 1 "single_thread": 表示该 kernel 只支持单线程使用, 可用的 treg 为 160 个
- 2 "symmetric_multi_thread": treg 在两个线程对称使用, 每个线程使用 80 个
- 3 "asymmetric_multi_thread": treg 在两个线程非对称使用, 目前只支持 [160, 0] 的分配方式
- 4 默认模式: "symmetric_multi_thread"

代码示例 (非对称使用模式):



1. 需要保证**线程0**使用 ≤ 160 个treg, **线程1**不使用treg
2. 相关JIRA: [\[S1SW-1679\] 编译器增加对非对称寄存器使用的自动化支持 - Jira](#) (已完成, 不再需要手动添加汇编配置)
3. 需要注意, 该模式下, kernel执行结束时, 由runtime主动添加了tsync指令来保证同步性, 已经消耗了sync id 0, 如果线程内代码逻辑需要使用tsync进行同步, 请使用非0 sync id值

代码块

```
1  __attribute__((treg_threading_mode("asymmetric_multi_thread")))  
2  __global__ void kernel(void) {  
3  int tregbase = 0;  
4  int tregsize = (!threadIdx.x) * 160;  
5  __asm__ volatile ("tcsr.w.r.b32 %0, 0x250" : : "r"(tregbase));  
6  __asm__ volatile ("tcsr.w.r.b32 %0, 0x254" : : "r"(tregsize));  
7  tsync_tb_sync(0);  
8  ace_bsync();  
9  
10     // TASK.....  
11  
12  tsync_tb_sync(0);  
13 }
```

功能验证, 可以通过汇编指令获取当前线程的tregbase与tregsize:

代码块

```
1  int tregbase = 0;  
2  int tregsize = 0;  
3  __asm__ volatile ("tcsr.r.b32 %0, 0x250" : "=r"(tregbase) : );  
4  __asm__ volatile ("tcsr.r.b32 %0, 0x254" : "=r"(tregsize) : );  
5
```



```
6  sipu::printf("tid: %d, tregbase: %d, tregsize: %d\n", threadIdx.x, tregbase, tregsize);
```

非对称TREG使用模式下，tregbase与tregsize均为0，即解除所有运行时限制，TREG需要由使用者负责其正确性及有效性。

6.17 RISC-V half数据类型指定

kernel代码中，使用了RISC-V Core，需要主动指定后续参与运算的数据类型（half），主要用于区分bfloat16与float16，指定方法如下：

```
代码块
1  // kernel代码中，后续参与运算的HALF，使用float16数据类型
2  __andescore_fp_mode(FP16);
3
4  // kernel代码中，后续参与运算的HALF，使用bfloat16数据类型
5  __andescore_fp_mode(BF16);
```

6.18 PE间的share mem互访

sipu支持Cooperative Launch，是类似CUDA Distributed Shared Memory的特性，实现了同一个cluster内PE间的share mem互访。

下表中mode 0为普通模式，仅支持PE内的share mem互访；mode 1和3是数据流常用的模式，需要提前在host上分配share mem；mode 2为本节所采用的互访模式，在kernel内分配share mem。

	chip scope	scope (based on PE/PEC/Chip)	chip id	logical pec id	logical pe id	备注
Shared Memory Space	Local chip L2B	mode 0: TB shared, PE Local (只PE可用)	N(0)	N(0)	N(0)	shared memory 模式 (单block)，HVM, TLSU需要这个range来选择本地PE还是出PE, range可配
		mode 1: Host allocated, global scope, used PE local	N(0)	N(0)	N(0)	数据流模式，Host分配和管理，给本PE使用
		mode 2: TBC shared, PEC Local (只PE可用)	N(0)	N(0)	Y	distributed shared memory 模式 (cluster)，cluster内，需要PE id
		mode 3: Host allocated, global scope, used by PE Peer	Y*	Y	Y	数据流模式，Host分配和管理，map到其他PE访问 (host or kernel)，
	Remote chip L2B	mode 3: used for remote chip	Y	Y	Y	数据流模式，片间，显式管理

用法要求如下：

1. 启动kernel的方式为cooperative launch，有新旧API如下：

```
代码块
1  // 方法1
```

```

2 void *args[] = {&C, &A, &B, &size};
3 sipuLaunchCooperativeKernel((void *)tile_dot_bf16_v2, 4, 4, 2, args, 0, 0);
4
5 // 方法2
6 auto device = sipu::Device::open(0);
7 sipuLaunchAttribute attributes[1];
8 attributes[0].id = sipuLaunchAttributeCooperative;
9 attributes[0].val.cooperative = 1;
10 sipuLaunchConfig_t config = {0};
11 config.gridDim = dim3{4, 1, 1};
12 config.clusterDim = dim3(4, 1, 1);
13 config.blockDim = dim3(2, 1, 1);
14 config.attrs = attributes;
15 config.numAttrs = 1;
16 sipuLaunchKernelExC(&config, (const void *)tile_dot_bf16_v2, args);
17

```

2. dim含义:

- Griddim: block总数; clusterIdx: 全局 cluster id
- Clusterdim: cluster 内启用的block数量; blockIdx: 全局 block id; blockIdxInCluster: cluster内的block id
- blockDim: block 内thread数量; threadIdx: block内的thread id

上例 4,4,2表示4 block in total, 4 block per cluster, 2 thread per block, 算下来使用了4/4=1 cluster。

- 对于本PE内的share mem可以直接访问, 其他PE的则需要用[地址map API](#)转换后访问。
- Sipu 1.5 仅支持tcore跨PE访问share mem。
- 需要使用blockIdxInCluster.x表示 cluster内的block索引

下例中实现了pe0和pe1的share mem累加, scalar_self = s[0][0]+s[1][0]+scalar_from_1

代码块

```

1 __shared__ float s[2][256];
2 float scalar_self = s[0][0];
3 if (threadIdx.x == 0) {
4     s[0][0] += s[1][0];
5 }
6 tsync_tbc_sync(0);
7 if (blockIdxInCluster.x == 0) {
8     float *s_from_1 =
9     cooperative_groups::cluster_group::map_shared_rank<float>(s[0], 1);
10    // float *s_from_1 = mappa2(s[0], 1);
11    tfloat32m1_t tile_from_1 = tld_blk_share_m1(s_from_1, 0);

```

```

11         float scalar_from_1 = tmv_r_f32(tile_from_1, 0);
12         scalar_self += scalar_from_1;
13     }
14     tsync_tbc_sync(0);
15
16     return scalar_self;

```

7 DTE使用方法

DTE的架构设计方案与使用说明，详细内容可参考：[📄 SiPU 1.0 DTE 架构设计方案 v1.2](#)

DTE可应用于多种场景，以提高数据搬运与运算的并行效率，下面就几种基础情况进行介绍。

7.1 Linear数据搬运

Linear数据搬运，只需要指定源数据位置（如global memory），目标数据位置（如shared memory），以及数据大小（字节数）即可，示例用法如下：

代码块

```

1  #define BUF_SIZE 1024
2
3  __shared__ uint8_t buf[BUF_SIZE];
4  tacp_linear((uint8_t *)src, BUF_SIZE, buf);
5  tacp_commit_group();
6
7  // 等待所有DTE操作完成
8  twait_tacp_cg(0);

```

7.2 Tile数据搬运

Tile数据搬运，搬运的内容为Tile Format数据，需要结合tensor map结构与坐标信息进行操作。

（1）tensor map为1024bit的结构体，结构体内容可参考DTE架构设计方案文档，其可通过create接口创建，填写关键信息即可，示例用法如下（具体可参考[📄 tensormap接口使用说明](#)文档，其中有对SltensorMap类型的详细介绍，同时runtime示例章节展示了完整的程序样例）：

代码块

```

1  // 注意：

```

```

2 // (1) tensor map的shape信息，与PyTorch的tensor shape信息，语义不同
3 //     tensor map的shape[0]具有物理排布上的连续性，与PyTorch相反
4 // (2) 目前runtime提供的tensor map创建接口，创建的内容位于host端
5 //     需要使用sipuMemcpy函数将其搬运至device端
6 // (3) siTensorMapEncodeLinear是默认的tensor map配置接口
7 //     核心参数由输入参数，通过决策树推断得出
8 //     数据shape相关的参数，为元素个数，非元素大小
9 // (4) tensor map encode有多个接口，分别适用不同场景
10 //     需要根据实际使用需求，进行判断选择
11 //     sipu::siTensorMapEncodeLinear
12 //     sipu::siTensorMapEncodeTiled
13 //     如果应用场景需要自定义tensor map的关键信息，需要使用完整参数配置接口
14 //     sipu::siTensorMapEncode
15
16 #include "sipu.h"
17
18 // globalDimInElements，原始数据的shape信息（以元素个数为单位）
19 // boxDimInElements，单次搬运数据的shape信息（以元素个数为单位）
20 uint32_t globalDimInElements[2] = {N, M};
21 uint32_t boxDimInElements[2] = {chunk_N, chunk_M};
22
23 sipu::SItensorMap tmap;
24 sipu::siTensorMapEncodeLinear(
25     &tmap,
26     sipu::SItensorMapDataType::TMAP_DTYPE_BF16,
27     2,
28     reinterpret_cast<uint8_t *>(src),
29     globalDimInElements,
30     boxDimInElements,
31     0
32 );
33
34 void *dev_tmap = nullptr;
35 sipuMalloc(&dev_tmap, sizeof(sipu::SItensorMap));
36 sipuMemcpy(dev_tmap, &tmap, sizeof(sipu::SItensorMap), sipuMemcpyHostToDevice);
37
38 kernel<<<1, 1, 1>>>(dev_tmap);
39
40 // tensor map使用结束后，需要释放相关资源
41 sipuFree(dev_tmap);

```

(2) 坐标信息为1024bit的数组（32个int），记录数据搬运的起始位置，示例如下：

代码块

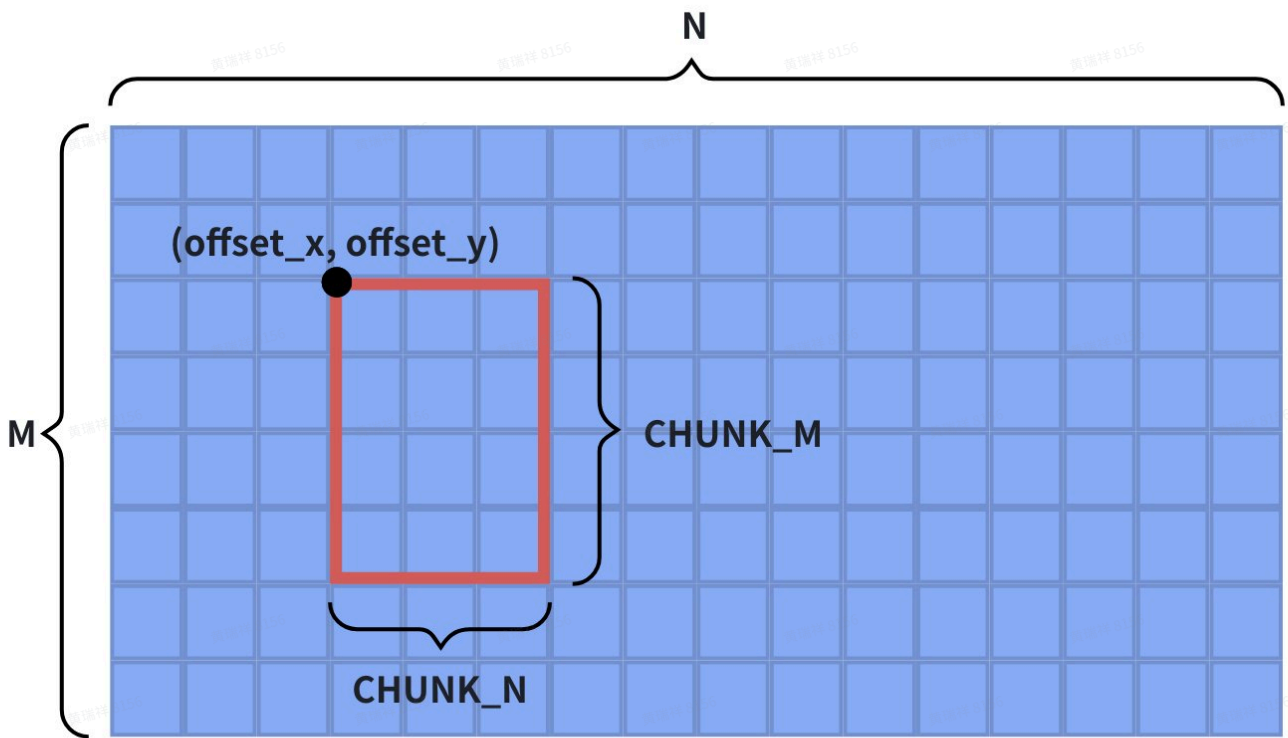
```

1 // 按照tensor的维度情况进行设定，此处以二维tensor为例
2 int32_t pos[32] = {0};
3 pos[0] = offset_x;
4 pos[1] = offset_y;

```

(3) 数据搬运

根据以上准备内容，从Tile Format tensor中搬运一块大小为 $\text{chunk_m} * \text{chunk_n}$ 的数据的方法如下：



代码块

```

1 #define BUF_SIZE (chunk_m * chunk_n * sizeof(DATATYPE))
2
3 // RVV用于配置信息的搬运
4 size_t vl = __riscv_vsetvl_e32m1(32);
5 vint32m1_t vec_tm = __riscv_vle32_v_i32m1(tm, vl);
6 vint32m1_t vec_pos = __riscv_vle32_v_i32m1(pos, vl);
7
8 __shared__ uint8_t buf[BUF_SIZE];
9
10 tacp_tile_srctm(vec_tm, vec_pos, buf);
11 tacp_commit_group();
12
13 // 等待所有DTE操作完成

```

```
14 twait_tacp_cg(0);
```

7.3 数据排布转换

由于TMAC单元执行MMA运算时，输入数据的排布形式需要为，某些情况下，原始数据的排布形式可能为Linear Format，此时就需要通过DTE来实现数据排布的转换（Linear Format -> Tile Format）。

数据排布转换，准备工作与Tile数据搬运的准备工作基本一致，区别在于搬运的API有差异，如下：

代码块

```
1 tacp_convert_srctm(vec_tm, vec_pos, buf);
```

8 同步功能介绍

相关同步接口builtin（单核同步，TB sync，TBC sync，Mbar）可参见[同步接口汇总](#)。

8.1 线程内同步（单线程同步）

SIPU内，存在多种类型的指令流水，且相互独立，因此在单个线程内部，也需要做好多个指令流水的同步操作。指令流水的结构框图，单核同步指令介绍，以及指令流水间的同步方案，具体内容可参考：[RV和Tile Core单核同步方案](#)

以一个具体场景为例：Tile Core写global memory，然后RV Core读取对应数据，为保证各模块读写数据的一致性，应有如下代码：

代码块

```
1 // Tile Core写global memory
2 tst_linear_global_m1(tile, out, 0U, 0L);
3
4 // 插入同步指令
5 twait_store_global(0); // 写入完成
6 ace_bsync(); // 指令同步
7 flush_l2c_wbinval_all(); // 刷新L2C缓冲区 (for RV Core, 后续可能不再需要)
8
9 // RV Core读取对应数据
10 float a = out[0];
```

对于不同场景下的指令流水间同步方案，有如下汇总：

producer\consumer	RV scalar/vector(L2B/Non-Cacheable DRAM access)	RV scalar/vector(Cacheable DRAM access)	Tile load/store	Tile DTE copy
RV scalar/vector(L2B access)	NA	NA	fence + 任意 Scalar load/store	fence + 任意 Scalar load/store
RV scalar/vector(DRAM access)	NA	NA	CCTL_cache_clean	CCTL_cache_clean
Tile load/store	twait.ls -> ace_bsync twait.mem -> ace_bsync twait -> ace_bsync	twait.ls -> ace_bsync -> CCTL_cache_invalidate twait.mem -> ace_bsync -> CCTL_cache_invalidate twait -> ace_bsync -> CCTL_cache_invalidate	twait.ls twait.mem twait twait -> ace_nbsync twait -> ace_bsync	twait.ls twait.mem twait twait -> ace_nbsync twait -> ace_bsync
Tile DTE copy	twait.tacp_cg -> ace_bsync twait.mem -> ace_bsync twait -> ace_bsync	twait.tacp_cg -> ace_bsync -> CCTL_cache_invalidate twait.mem -> ace_bsync -> CCTL_cache_invalidate twait -> ace_bsync -> CCTL_cache_invalidate	twait.tacp_cg twait.mem twait twait -> ace_nbsync twait -> ace_bsync	twait.tacp_cg twait.mem twait twait -> ace_nbsync twait -> ace_bsync

编译器与runtime已经对上述情况，提供了集成完毕的API，如示例中的3条插入的同步指令可更新为：

代码块

```

1  twait_store_global(0);           // 写入完成
2  ace_bsync();                     // 指令同步
3  flush_l2c_wbinval_all();         // 刷新L2C缓冲区（for RV Core，后续可能不再需要）
4  |
5  |
6  |
7  V
8  tst_rv_cache_fence(0);

```

更多具体内容可参考：[📖 同步接口汇总](#)

8.2 指令同步（多线程同步）

根据SIPU中的线程组织形式，多线程同步也可分为3个层次：

（1）block内线程同步

block内最多可配置2个线程，一般情况下，2个线程的指令执行是无序的，如下：

代码块

```
1  __global__ void test_kernel(void) {
2      uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
        blockDim.x + threadIdx.x;
3
4      for (uint32_t i = 0; i < 10; i++) {
5          printf("tid = %u, loop = %u\n", tid, i);
6      }
7  }
8
9  int main(int argc, char *argv[]) {
10     dim3 grid{1};
11     dim3 cluster{1};
12     dim3 block{2};
13
14     test_kernel<<<grid, cluster, block>>>();
15     sipuDeviceSynchronize();
16 }
```

2个kernel的多轮输出是无序交错的：

代码块

```
1  [sipu-core-001]: tid = 1, loop = 0
2  [sipu-core-001]: tid = 1, loop = 1
3  [sipu-core-001]: tid = 1, loop = 2
4  [sipu-core-001]: tid = 1, loop = 3
5  [sipu-core-001]: tid = 1, loop = 4
6  [sipu-core-000]: tid = 0, loop = 0
7  [sipu-core-000]: tid = 0, loop = 1
8  [sipu-core-001]: tid = 1, loop = 5
9  [sipu-core-000]: tid = 0, loop = 2
10 [sipu-core-001]: tid = 1, loop = 6
11 [sipu-core-000]: tid = 0, loop = 3
```



```
12 [sipu-core-001]: tid = 1, loop = 7
13 [sipu-core-000]: tid = 0, loop = 4
14 [sipu-core-001]: tid = 1, loop = 8
15 [sipu-core-000]: tid = 0, loop = 5
16 [sipu-core-001]: tid = 1, loop = 9
17 [sipu-core-000]: tid = 0, loop = 6
18 [sipu-core-000]: tid = 0, loop = 7
19 [sipu-core-000]: tid = 0, loop = 8
20 [sipu-core-000]: tid = 0, loop = 9
```

我们可以在每个轮次后面增加block内的线程同步接口，来实现指令同步：

代码块

```
1 __global__ void test_sync_kernel(void) {
2     uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
    blockDim.x + threadIdx.x;
3
4     for (uint32_t i = 0; i < 10; i++) {
5         printf("tid = %u, loop = %u\n", tid, i);
6         tsync_tb_sync(0);
7     }
8 }
```

输出结果如下：

代码块

```
1 [sipu-core-001]: tid = 1, loop = 0
2 [sipu-core-000]: tid = 0, loop = 0
3 [sipu-core-000]: tid = 0, loop = 1
4 [sipu-core-001]: tid = 1, loop = 1
5 [sipu-core-000]: tid = 0, loop = 2
6 [sipu-core-001]: tid = 1, loop = 2
7 [sipu-core-000]: tid = 0, loop = 3
8 [sipu-core-001]: tid = 1, loop = 3
9 [sipu-core-000]: tid = 0, loop = 4
10 [sipu-core-001]: tid = 1, loop = 4
11 [sipu-core-000]: tid = 0, loop = 5
12 [sipu-core-001]: tid = 1, loop = 5
13 [sipu-core-000]: tid = 0, loop = 6
14 [sipu-core-001]: tid = 1, loop = 6
15 [sipu-core-000]: tid = 0, loop = 7
16 [sipu-core-001]: tid = 1, loop = 7
17 [sipu-core-000]: tid = 0, loop = 8
18 [sipu-core-001]: tid = 1, loop = 8
```

```
19 [sipu-core-000]: tid = 0, loop = 9
20 [sipu-core-001]: tid = 1, loop = 9
```

可见，2个线程的每个轮次，都是同步执行完毕后，再进入下一个轮次的。

除了多线程同时等待同步指令`tsync_tb_sync`，SIPU还提供了线程协作指令`tsync_tb_arrive`，和同步指令搭配使用，可以实现更灵活的同步形态，示例如下：

代码块

```
1 __global__ void test_arrive_kernel(void) {
2     uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
    blockDim.x + threadIdx.x;
3
4     for (uint32_t i = 0; i < 8; i++) {
5         printf("tid = %u, loop = %u\n", tid, i);
6         if (tid == 0) {
7             tsync_tb_sync(i);
8         } else {
9             tsync_tb_arrive(i);
10        }
11    }
12 }
```

此时输出内容为：

代码块

```
1 [sipu-core-001]: tid = 1, loop = 0
2 [sipu-core-001]: tid = 1, loop = 1
3 [sipu-core-001]: tid = 1, loop = 2
4 [sipu-core-001]: tid = 1, loop = 3
5 [sipu-core-000]: tid = 0, loop = 0
6 [sipu-core-001]: tid = 1, loop = 4
7 [sipu-core-001]: tid = 1, loop = 5
8 [sipu-core-000]: tid = 0, loop = 1
9 [sipu-core-001]: tid = 1, loop = 6
10 [sipu-core-000]: tid = 0, loop = 2
11 [sipu-core-001]: tid = 1, loop = 7
12 [sipu-core-000]: tid = 0, loop = 3
13 [sipu-core-000]: tid = 0, loop = 4
14 [sipu-core-000]: tid = 0, loop = 5
15 [sipu-core-000]: tid = 0, loop = 6
16 [sipu-core-000]: tid = 0, loop = 7
```

可见，采用sync接口的线程0，需要等待相同标号（轮次）的线程1完成后，才能继续执行，而使用arrive接口的线程1，只是通过该接口通知线程0，而不做等待动作（非阻塞）。

(2) cluster内线程同步

cluster内的线程同步，与block内的线程同步类似，区别在于将同步的线程范围扩大到单个cluster（最多8个线程）。SIPU也提供了2个指令用于做线程同步：

代码块

```
1  __global__ void test_tbcsync_kernel(void) {
2      uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
        blockDim.x + threadIdx.x;
3
4      for (uint32_t i = 0; i < 8; i++) {
5          printf("tid = %u, loop = %u\n", tid, i);
6          tsync_tbc_sync(0);
7      }
8  }
9
10 __global__ void test_tbcarrive_kernel(void) {
11     uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
        blockDim.x + threadIdx.x;
12
13     for (uint32_t i = 0; i < 8; i++) {
14         printf("tid = %u, loop = %u\n", tid, i);
15         if (tid == 0) {
16             tsync_tbc_sync(i);
17         } else {
18             tsync_tbc_arrive(i);
19         }
20     }
21 }
```

线程同步范围扩大后，同步的代价也会增加。需要权衡评估同步的必要性与收益，做好cluster内的功能设计。

(3) device内线程同步

SIPU并没有提供全局的线程同步方法，如果可以实现类似功能，可以结合atomic指令与逻辑判断来实现，示例如下：

代码块

```
1  __global__ void test_atomic_kernel(void *mem) {
```

```

2      uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
      blockDim.x + threadIdx.x;
3
4      uint32_t *data = (uint32_t *)mem;
5      for (uint32_t i = 0; i < 8; i++) {
6          while (1) {
7              if (tld_atom_addu32_global(data, 0) == i) break;
8          }
9          printf("tid = %u, loop = %u\n", tid, i);
10         if (tid == 0) {
11             tst_atom_addu32_global(data, 1);
12         }
13     }
14 }

```

示例中，由单个线程（thread 0）通过atomic store对原始数据进行自增操作，然后所有线程通过atomic load对原始数据进行访问和判断，来达到device级别的线程同步。

8.3 访存同步

对于SIPU内多个pipeline的访存控制的流程介绍，可以参见：[RV和Tile Core单核同步方案](#)

(1) LSU单元同步

SIPU提供了以下指令来完成LSU单元的同步操作：

代码块

```

1 void twait_load_global(uint32_t op0);
2 void twait_load_share(uint32_t op0);
3 void twait_store_global(uint32_t op0);
4 void twait_store_share(uint32_t op0);

```

其中，op0参数表示load/store指令剩余多少数量后才能让twait指令提交，一般情况下，op0参数填写为0，来保证前序的load/store访存动作均已完成。

如果在开发过程中，存在RAW（read after write）或WAR（write after read）行为的话，是需要显式调用以上相关twait指令，来保证访存完成的。

(2) DTE单元同步

SIPU提供了以下指令来完成DTE单元的同步操作：

代码块

```
1 void twait_tacp_cg(uint32_t op0);
```

其中，op0参数表示tacp指令中commit group的数量，该指令会在ACE Pipeline中执行，在tacp_cnt小于等于指令中的op0时提交，允许后续指令执行。一般情况下，op0参数填写为0，可保证前序的DTE搬运动作均已完成。

(3) 全同步

SIPU提供了以下指令来完成所有访存单元的同步操作：

代码块

```
1 void twait_mem();
```

所有访存指令（包括所有的TACP指令）全部执行完成后才可以提交。

8.4 memory barrier



需要明确已知mbarrier所在的shared memory（L2B）位置为本地或远程，以使用正确类型的mbarrier（显示指定scope）：

// Backward-compatible alias: existing code uses sipu::mbarrier (local scope)

using mbarrier = barrier<barrier_scope::local>;

// New alias for cross-chip barrier operations

using remote_mbarrier = barrier<barrier_scope::remote>;

可参考如下示例：

代码块

```
1 __shared__ sipu::mbarrier mbar_local;
2 __shared__ sipu::remote_mbarrier mbar_remote;
3
4 int local_pid;
5 int remote_pid;
6
7 mbar_local.init(1);
8 mbar_local.increase(2);
9 mbar_local.arrive();
10 mbar_local.wait(local_pid);
11
12 mbar_remote.init(1);
13 mbar_remote.increase(2);
```

```
14 mbar_remote.arrive();
15 mbar_remote.wait(remote_pid);
```

Memory barrier是存在shared memory中或global memory中的数据结构体，用来实现多线程之间访问memory的同步。与Tsync类似，需要参与的线程都arrive后才能wait成功，相对于Tsync，mbar还允许用户操作其内部的同步计数器，以及允许DTE参与同步。详见[SiPU 1.5 Sync 多核同步机制架构设计方案 v0.8](#)。

1. Memory Barrier (Mbar)是创建于L2B (Shared Memory)上的一种barrier，用于该Shared Memory可见范围内的Thread之间的同步，理论上可创建的Mbar数量只受L2B总大小限制。
2. 从用途上，Mbar可分为两种：
 - a. 进行线程同步。
 - b. 追踪写L2B指令的完成状态，并通过**联合指令**和**代理模块**（目前仅支持DTE）的支持完成硬件自动同步。
3. 从同步范围上，可以支持：
 - a. 1个Thread内DTE指令和后续消费指令之间的同步。（也可用bulk_group替代）
 - b. 1个TB内部的2个threads之间进行同步。
 - c. 1个TBC内部的 ≤ 8 threads之间的同步。
 - d. 1个kernel内部的 ≤ 128 threads之间的同步。
 - e. Multi-kernel(Chip/Multichip)中任意subset of threads之间的同步。其中跨TBC，跨Kernel的同步需要kernel cooperative launch。
4. Mbar需要Programmer显式创建，显式销毁。在创建之后和销毁之前需要同步。
 - a. 用于TBC范围内同步时，可以直接使用tsync.tb/tbc.sync进行同步。
 - b. 用于跨TBC范围同步时，需要使用global memory atomic同步，同时需要kernel下发之前将对atomic地址初始化为0。
5. Mbar相比TSync。
 - a. 数量基本不受限制。（只受到L2B空间限制）
 - b. 可以更为灵活的指定参与threads。
 - c. DTE copy（TACP）可以异步执行并对Mbar进行更新，producer侧效率更高。
6. 支持Split同步原语。实际上包含了arrive/wait两步同步原语，和semaphore类型的inc/dec同步原语。
 - a. `mbar_arrive`：Non-blocking操作。表示Executing Threads到达该barrier（的某个phase），但不阻塞其后续指令执行，可继续执行无关操作。

- b. `mbar_wait` : Blocking操作, 将导致Executing Threads被阻塞在该barrier (的某个phase), 直到barrier满足完成条件, Thread可立即继续执行后续指令。
- c. `mbar_inc_tx_cnt` : Non-blocking操作。由thread执行, 增加某个mbar的tx_cnt数。不阻塞对应的thread/DTE后续执行。
 - i. 可用于给DTE数据搬运设置需要搬运的byte数。
- d. `mbar_dec_tx_cnt` : Non-blocking操作。
 - i. 可由thread执行, 减去某个mbar的tx_cnt数。不阻塞对应的thread/DTE后续执行。
 - ii. 可由DTE在完成一笔DTE Copy时代为执行, 表示发起该笔DTE Copy的Thread完成数据搬运Byte数, 但不阻塞对应的thread/DTE后续执行。

以PE内2个线程参与同步为例, 有如下代码:

(1) 双线程协作, 单线程打卡

代码块

```
1  #include <siipu_runtime.h>
2  #include <siipu.h>
3  #include <stdio.h>
4
5  #define N 128
6  #define STEP 16
7  #define LOOP (N / STEP)
8
9  __global__ void test_larrive_kernel(void *mem) {
10     uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
        blockDim.x + threadIdx.x;
11     int *data = (int *)mem;
12     long copy_size = STEP * sizeof(int);
13
14     __shared__ int buf[STEP];
15     __shared__ siipu::mbarrier mbar_acq;
16     __shared__ siipu::mbarrier mbar_rel;
17
18     if (tid == 0) {
19         mbar_rel.init(1);
20     } else {
21         mbar_acq.init(1);
22     }
23     siipu::tb::sync_obj_0::sync();
24
25     int acq_phase_id = 0;
26     int rel_phase_id = 0;
27
28     if (tid == 0) {
```

```

29     mbar_rel.arrive();
30 }
31
32 for (int i = 0; i < LOOP; i++) {
33     if (tid == 0) {
34         mbar_rel.wait(rel_phase_id);
35         rel_phase_id++;
36         mbar_acq.increase(1);
37         tcp_linear_mbar((const uint8_t *) (data + i * STEP), copy_size,
38 (const uint8_t *)&mbar_acq, (uint8_t *)buf);
39         mbar_acq.arrive();
40     } else {
41         mbar_acq.wait(acq_phase_id);
42         acq_phase_id++;
43         for (int j = 0; j < STEP; j++) {
44             sipu::printf("tid = %u, loop = %d, id = %d, val = %d\n", tid,
45 i, j, buf[j]);
46         }
47         mbar_rel.arrive();
48     }
49 }
50
51 int main(int argc, char *argv[]) {
52     int data[N] = {0};
53     for (int i = 0; i < N; i++) {
54         data[i] = i;
55     }
56
57     void *mem = nullptr;
58     sipuMalloc(&mem, N * sizeof(int));
59     sipuMemcpy(mem, data, N * sizeof(int), sipuMemcpyHostToDevice);
60
61     dim3 grid{1};
62     dim3 cluster{1};
63     dim3 block{2};
64
65     test_larrive_kernel<<<grid, cluster, block>>>(mem);
66     sipuDeviceSynchronize();
67
68     sipuFree(mem);
69
70     return 0;
71 }

```


(2) 多线程协作，多线程打卡

代码块

```
1  #include <sipu_runtime.h>
2  #include <sipu.h>
3  #include <stdio.h>
4
5  #define N 128
6  #define STEP 16
7  #define LOOP (N / STEP)
8
9  __global__ void test_mbar_kernel(void *mem) {
10     uint32_t tid = clusterIdx.x * clusterDim.x * blockDim.x + blockIdx.x *
        blockDim.x + threadIdx.x;
11     int *data = (int *)mem;
12     long copy_size = STEP * sizeof(int);
13
14     __shared__ int buf[STEP];
15     __shared__ sipu::mbarrier mbar_acq;
16     __shared__ sipu::mbarrier mbar_rel;
17
18     if (tid == 0) {
19         mbar_rel.init(2);
20     } else {
21         mbar_acq.init(2);
22     }
23     sipu::tb::sync_obj_0::sync();
24
25     int acq_phase_id;
26     int rel_phase_id = mbar_rel.arrive();
27
28     for (int i = 0; i < LOOP; i++) {
29         if (tid == 0) {
30             mbar_rel.wait(rel_phase_id);
31             mbar_acq.increase(1);
32             mbar_acq.arrive();
33             tcp_linear_mbar((const uint8_t *) (data + i * STEP), copy_size,
                (const uint8_t *)&mbar_acq, (uint8_t *)buf);
34         } else {
35             acq_phase_id = mbar_acq.arrive();
36             mbar_acq.wait(acq_phase_id);
37             for (int j = 0; j < STEP; j++) {
38                 sipu::printf("tid = %u, loop = %d, id = %d, val = %d\n", tid,
                    i, j, buf[j]);
39             }
40         }
```

```

41
42     rel_phase_id = mbar_rel.arrive();
43 }
44 }
45
46 int main(int argc, char *argv[]) {
47     int data[N] = {0};
48     for (int i = 0; i < N; i++) {
49         data[i] = i;
50     }
51
52     void *mem = nullptr;
53     sipuMalloc(&mem, N * sizeof(int));
54     sipuMemcpy(mem, data, N * sizeof(int), sipuMemcpyHostToDevice);
55
56     dim3 grid{1};
57     dim3 cluster{1};
58     dim3 block{2};
59
60     test_mbar_kernel<<<grid, cluster, block>>>(mem);
61     sipuDeviceSynchronize();
62
63     sipuFree(mem);
64
65     return 0;
66 }
67

```

8.5 同步相关的device function

需要注意的是，原始的同步指令，如`tsync_tb_sync()`与`tsync_tbc_sync()`只是保证多线程的tile core的同步，并没有包含memory同步以及tile core与rv core之间的指令同步含义，为了实现更加完整的同步行为，runtime封装了常用的同步相关的device function，详细描述见 [同步接口汇总](#)。

例如类似cuda的线程块内同步，有对应的`__syncthreads()`实现，其中封装了rv core的同步与访存同步的功能。

9 性能分析方法

算子开发完成后，如果通过测试用例验证了其功能的正确性，那么对其性能进行分析与优化，则是提升系统整体运行效率的关键步骤。

在芯片未回片的情况下，通常会使用各种仿真工具进行功能验证与性能分析，包括perf model，QEMU，RTL仿真验证等，下面对目前可用的工具情况进行介绍。

算子基本信息的查看，也可通过定制化的readelf等工具来实现，比如查看算子的寄存器、L2B等的占用情况：

代码块

```
1  # 定制readelf工具的位置，可将sipu_sdk_latest字段替换为想要使用的sdk版本
2  /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/bin/nds64le-elf-newlib-
   v5d/bin/llvm-readelf
3
4  # 查看算子编译得到的elf文件的基本信息
5  llvm-readelf *_sipu_fatbin.elf -p .sipu.resource
6
7  # 对于多个source (su) 文件编译生成的elf文件，需要增加--sipufatbin-multi参数来获得每个
   elf的信息
8  # 对于不清楚elf是否由多个source生成的情况，建议全部增加--sipufatbin-multi参数，来避免
   获取信息遗漏
9  llvm-readelf *_sipu_fatbin.elf --sipufatbin-multi -p .sipu.resource
10
11 # 也可以直接看编译目录中的相关文件，形如：
12 cat build/_SipuWork.*/**/*_sipu_fatbin.llvm.resource
13
14 # 输出信息一般如下所示
15 String dump of section '.sipu.resource':
16 [    0] _Z14*_ii.treg = 2
17 [   80] _Z14*_ii.sram = 0
18 [  100] _Z14*_ii.tbc_sync = 0
19 [  184] _Z14*_ii.tb_sync = 0
20 [  208] _Z14*_ii.cooperative_enable = 0
21 [  294] _Z14*_ii.stack = 256
```

9.1 ESL Model

ESL (Electronic System Level) Model是用于评估算子执行功能与性能的重要依赖，SIPU ESL Model提供了function与performance两种模式，区别在于Tile Core的实现差异，两种模式的配置与跑测方法是类似的，更多具体内容可参考文档：[SiPU ESL Model](#)



需要注意：

1. ESL Model仿真评估过程中，需要保持SDK和Python package的版本一致性，否则可能会导致分析报错或执行中断。

2. 如果更新了ESL Model的SDK，一般也需要重新安装Python package依赖（如下）

```
pip install --force-reinstall  
/share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/lib/*.wh  
l
```

3. esl_model仿真目前不支持在kernel内部printf，需要注释掉kernel内部的printf，否则会报错如下：

```
Fault (breakpoint, 0x5)*** Could not read pc!
```

```
cfgroot stop is driven
```

```
[AndeSysC] core1 done, 1 core still running
```

```
Info: /OSCI/SystemC: Simulation stopped by user.
```

```
terminate called after throwing an instance of  
'sc_core::sc_report'
```

```
what(): Error: (E546) sc_start called after sc_stop has  
been called
```

```
In file:
```

```
/jenkins/workspace/sipu_sdk_release_daily_395/sipu_sw/systemc-  
2.3.4/src/sysc/kernel/sc_simcontext.cpp:1700
```

代码块

```
1  # 配置仿真模型类型  
2  # 如果需要切换回基础的arch model，可使用：  
3  # export SIRT_SHIM_CMODEL_TYPE=ARCH_MODEL  
4  export SIRT_SHIM_CMODEL_TYPE=ESL_MODEL  
5  
6  # 安装依赖package  
7  pip install --force-reinstall  
  /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/lib/*.whl  
8  
9  # 生成license，此过程需要ifconfig，可在系统内预先安装net-tools工具  
10 bash /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/bin/setup_mac.sh  
11  
12 # 拷贝配置文件至程序执行目录，并更新文件名称为eslmodel.toml  
13 # 根据需要选择func配置或perf配置  
14 cp  
  /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/lib/eslmodel.func.toml  
  eslmodel.toml
```

```

15 # 或
16 cp
   /share_data/sipu_sdk_debug/sipu_sdk_latest/sipu_sdk_rel/lib/eslmodel.perf.toml
   eslmodel.toml
17
18 # 修改配置文件eslmodel.toml，如下（super mode为1表示PEG level）：
19 statistics_on = true
20 counter_sampling_on = true
21 tfe_commit_trace_level = 1
22 super_mode = 1           # 1个PEG
23 active_pec_mask = 0xf     # 4个PEC
24 active_pe_mask = 0xffff   # 16个PE

```

仿真环境配置完毕后，可以开始进行待分析算子的跑测了，示例如下：

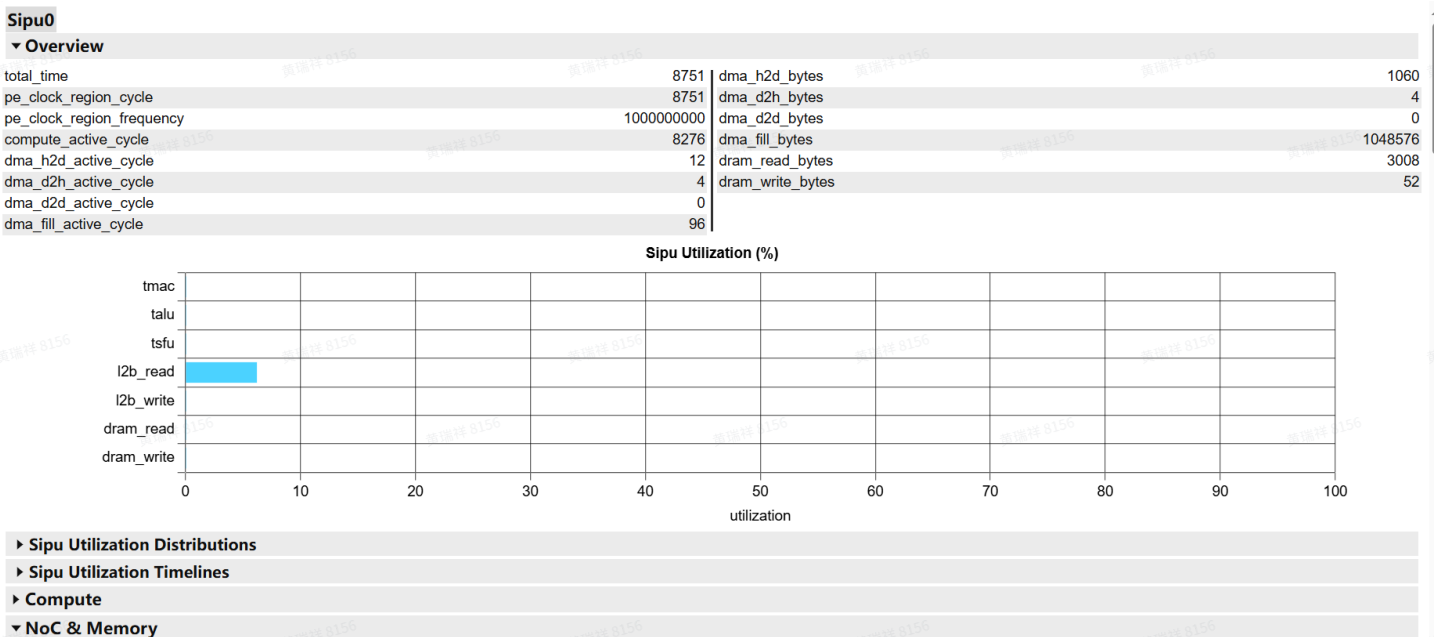
代码块

```

1 # 在部署了配置文件eslmodel.toml的目录下，运行可执行程序，如test_host
2 ./test_host
3
4 # 可执行程序执行完毕后，运行Python模块
5 # 可能需要预先安装tomlkit
6 python -m esl_model.esl_statistics_gen
7 python -m esl_model.report.html.esl_report
8
9 # Python模块运行完成后，会有本地地址输出，可将该地址拷贝至浏览器进行访问，形式如下：
10 Serving EsL Report on 10.193.64.54:34239

```

浏览器访问相关地址后，展示的性能分析图表如下所示：



可以全面浏览分析算子执行过程中的访存与流水线，并考虑是否有性能改善的方案。

在performance模式下，更完整的介绍内容可参考 [Perf Model使用参考](#)。

9.2 RTL仿真

RTL仿真是在真实的硬件环境中进行算子的实测，并输出跑测结果。RTL仿真环境的使用，可参考文档：

- (1) [IT: 1-用户首次登陆ETX](#)
- (2) [PE DV 算子运行及分析方法](#)

在ETX设备上，当RTL仿真执行完成后，会在对应目录下生成一系列文件，可通过浏览器以本地文件浏览方式，直接打开该目录内的html文件，即可查看指令执行情况与统计信息。

9.2.1 波形查看

HTML页面对于指令执行的细节，显示的信息较为有限，可通过查看波形，分析指令最完整的流水过程，查看波形需要使用verdi工具，verdi工具一般通过bsub启动，如下：

代码块

```
1 bsub -Is verdi ***
```

下面简要介绍verdi工具的相关参数，主要介绍与查看波形相关的内容：

代码块

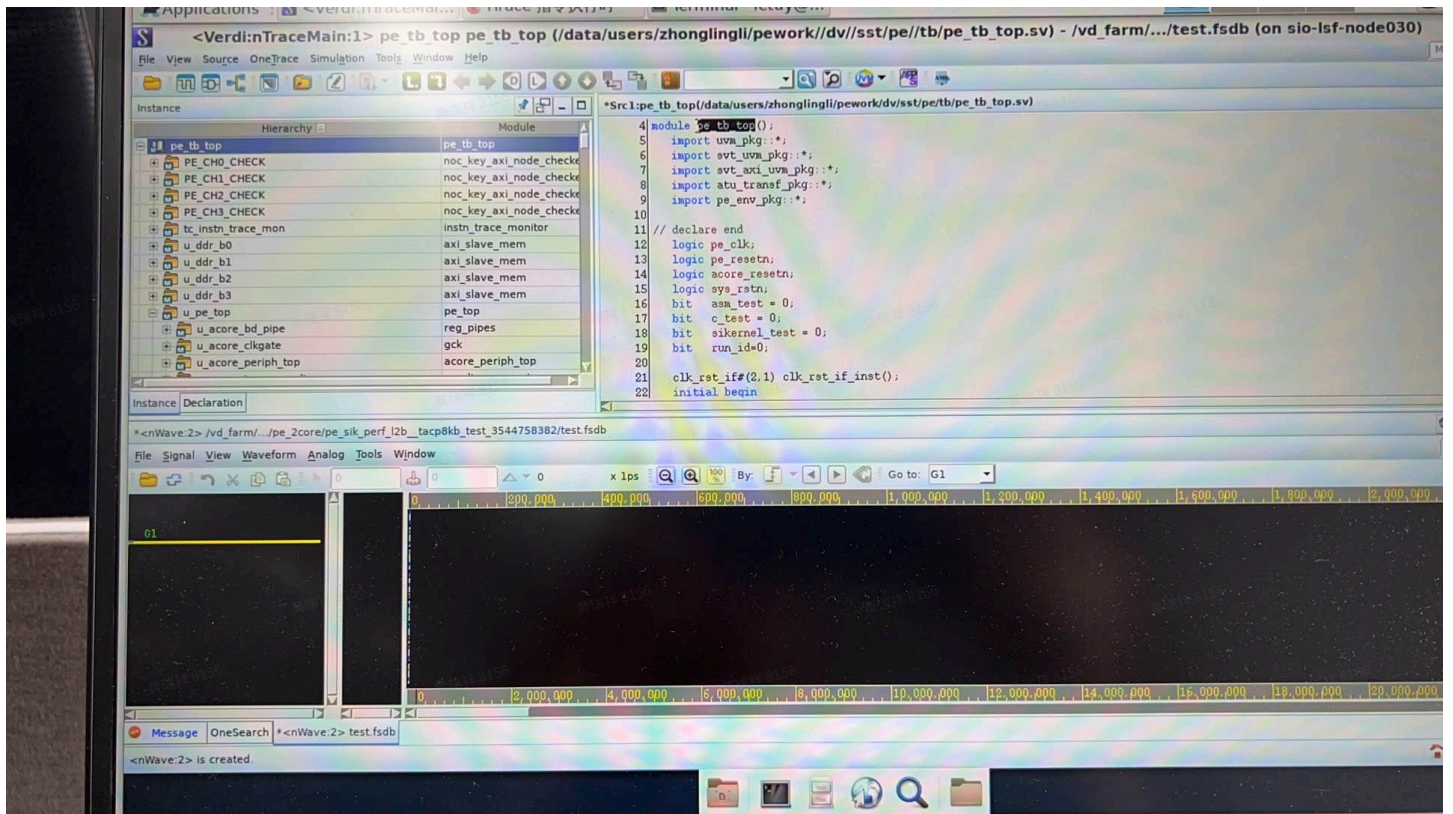
1	-h	查看帮助信息
2	-ssf	加载波形文件，如.fsd, .vf等
3	-sswr	加载波形restore文件（.rc文件）
4	-simflow	加载VCS生成的Knowledge Database（KDB）
5	-dbdir simv.daidir/	打开仿真器数据库（database）文件夹

典型的波形查看指令如下：

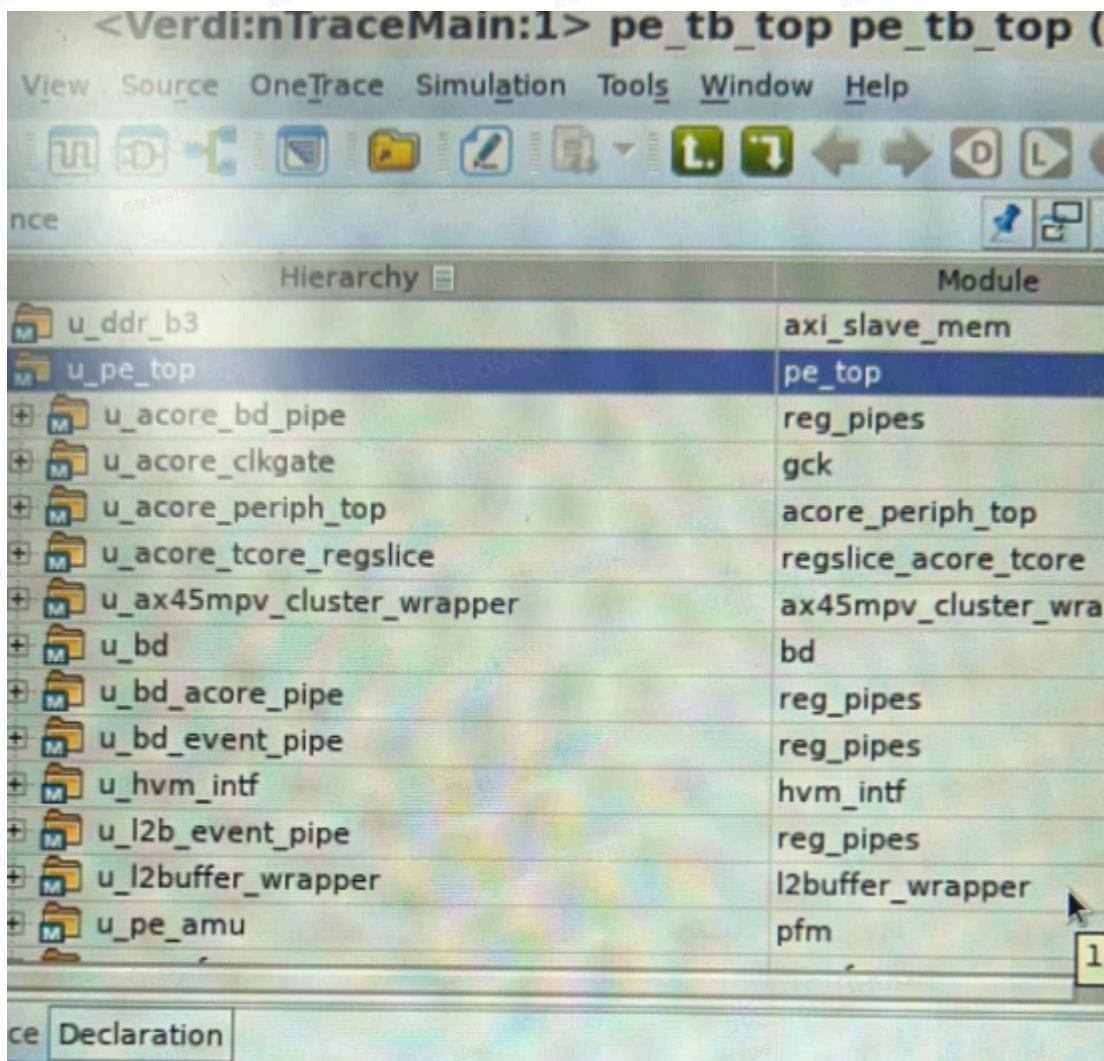
代码块

```
1 bsub -Is verdi -dbdir libs/pe_2core/elab/simv.daidir/ -ssf pe_2core/算子目录/test.fsd &
```

verdi启动后，终端显示界面如下所示：

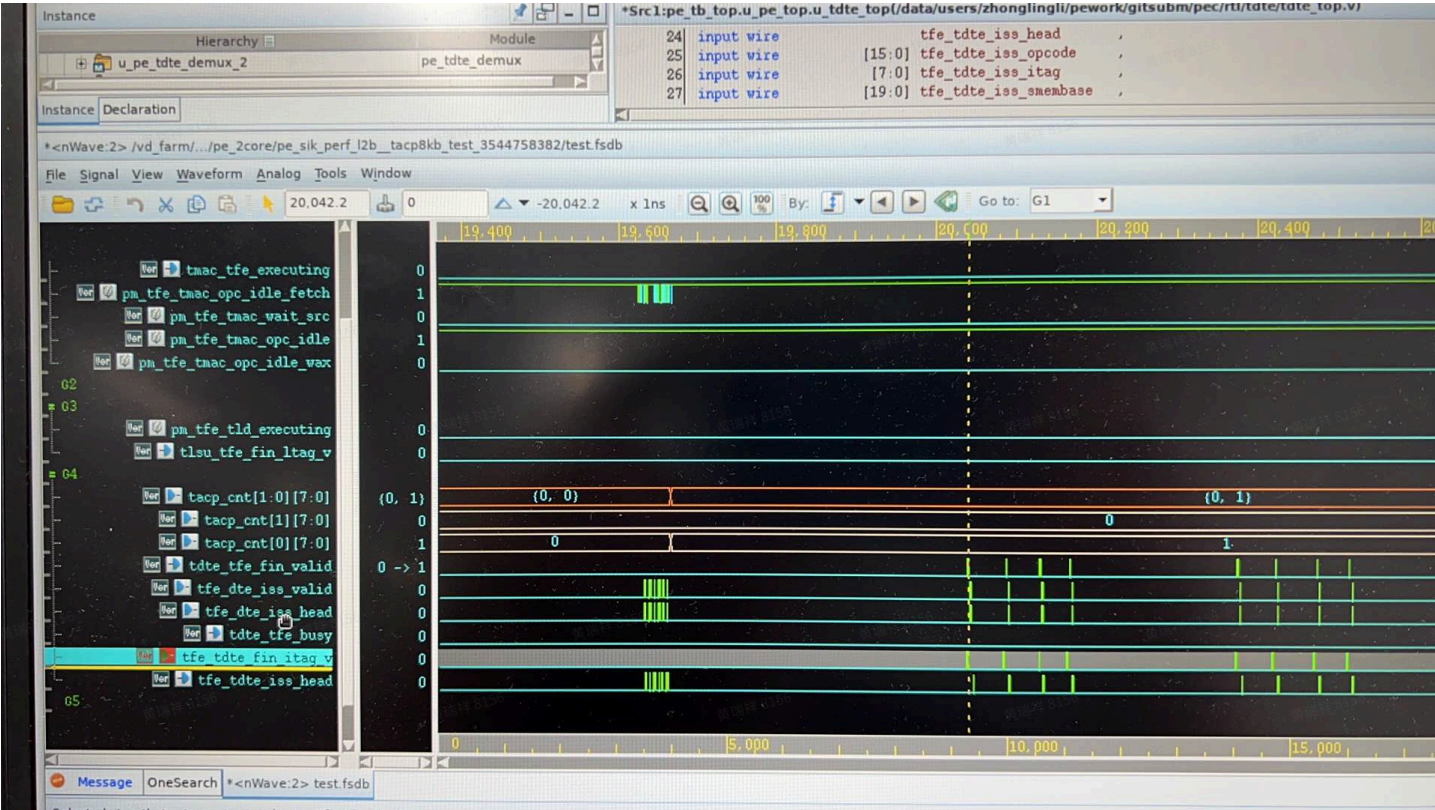


左侧的pe_tb_top，内部以硬件的组成结构分布，我们主要关注的是单个PE内部的指令执行情况，如下：



u_pe_top中, 包含acore, tcore, dte, lsu等基本组成单元, 可以从关注的单元内部, 寻找关键信号, 拖动到下方波形栏中进行观察分析。对于指令执行过程中产生的信号, 可以参考该文档: [TFE MAS V0.9](#)

拖动关键信号后, 得到的波形图如下所示:



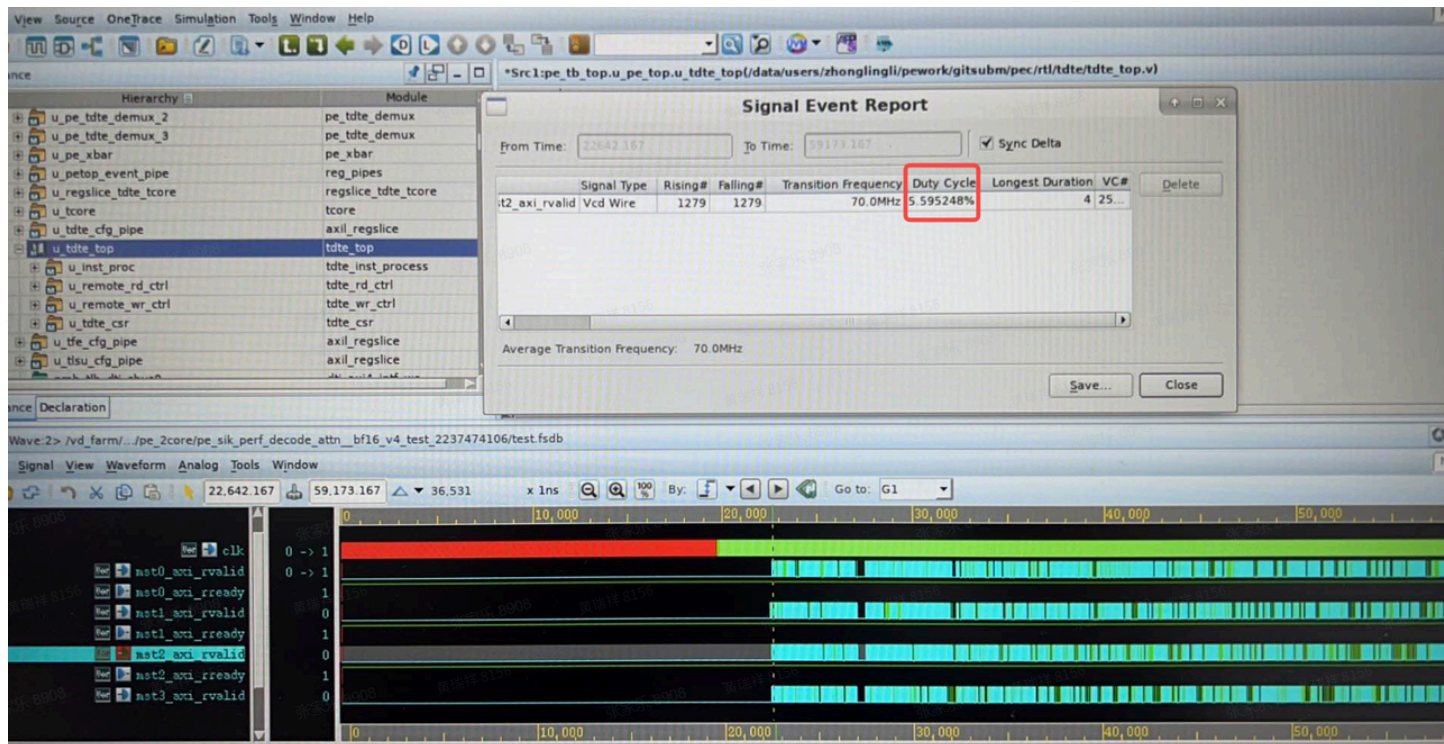
常用信号, 也可以通过波形栏的File-Save Signal来保存, 后续遇到分析同类型的信号时, 可以通过File-Restore Signal来加载分析。

9.2.2 信号占空比查看

在波形主页中, 通过“鼠标左键”确定关注信号的时间轴起点(黄色竖线), 通过“鼠标滚轮”确定关注信号的时间轴终点(白色竖线), 然后访问View->Signal Event Report, 在弹出的对话框中, 即可看到该信号的:

- (1) 上升沿次数
- (2) 下降沿次数
- (3) 占空比

如下图所示:



也可以同时观察多个信号的占空比情况，将关注信号拖入对话框中即可。

以TMAC单元执行效率分析为例，可以参考文档：[国DV仿真结果分析方法](#)

9.2.3 常用信号汇总

所属单元	信号名称	信号说明	路径
PE	clk_pe	时钟	
TMAC	tfe_trf_tmac_tr_last	表示tmac的源操作数ready	
	tmac_executing	TMAC在工作，目标：该信号尽量持续高电平	
	pm_tfe_tmac_	TMAC在等待参与运算的数据就位	
	tmac_wr_bkpressure	TMAC写结果被反压	
	isq_opc_tmac_tid	tmac被PE内部的哪个线程占用，类似的tid还有alu sfu DTE LSU等	
TFE	wb_ctl_wb_en	Write back使能：所有指令写回寄存器	
	wb_ctl_wb_trid[8:1]	Write back：Tile Reg id	
	wb_ctl_wb_trid[0]	Write back标志位：0-低512B，1-高512B	

DTE	tfe_dte_iss_valid/head	发起dte搬运	
	tfe_tdte_fin_itag_v	完成dte搬运	
	mst0/1/2/3_axi_arvalid	dte内部axi发起搬运	
	mst0/1/2/3_axi_rvalid	dte内部axi完成搬运	
	rmt_p_rd_ots_cnt[0/1/2/3]	Read outstanding count	
	tdte_idle	DTE单元是否空闲	
	pending_tacp_cnt	tccore内pending的tacp cmt-group tile-instr的数量	
	tacp_cnt[0/1]	单个thread的tacp任务数量，[0/1]表示tid	
LSU	pm_tfe_tld_executing	lsu正在load	
	tlsu_tfe_fin_ltag_v	lsu有load完成	
	tfe_tlsu_iss_valid	TLSU接收到指令	
	pm_tfe_tst_executing	lsu正在store	
Acore->Tcore	twait_tacp_busy	acore-ace流水线里面有一条twait.cmg_grp cnt阻塞了，会导致tile指令无法下发到tile-core	
TALU	pm_tfe_talu_executing	talum在工作	
	talum_tfe_td_v	talum完成运算，发起writeback请求（代表talum result 写到 talum-opc了，至于啥时候写到tile regfile是由tfe调度决定，需配合wb_ctl_wb_trid确认写回reg的时刻）	
TSFU	pm_tfe_tsfu_executing	TSFU在工作	
	tsfu_tfe_td_v	同TALU	
RVV	slv_rvmmio_axi_arvalid	读DRAM	
	slv_rvmmio_axi_awvalid	写DRAM	

slv_hvm_arvalid	读L2B	
slv_hvm_awvalid	写L2B	

9.3 EMU仿真

具体操作流程，以及波形的抓取与查看分析方法，可参考文档：[📄EMU平台仿真流程](#)

9.3.1 常用信号汇总

由于EMU环境全波形抓取，占用空间大（数十GB），且需要进行格式转换（ztdb to zwd），转换时间长（数小时），所以考虑仅针对关键信号进行波形抓取。

此处梳理的信号，与9.2.3一节中的内容对应，将其转换为DE环境中的信号名称，并提供给HAV团队进行版本的制作。

目前关注的所有信号，均位于以下路径：

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top

且全部集中在该路径的u_tcore与u_tdte_top两个模块中，所以可以考虑在制作限定信号的版本时，指定抓取这两个模块的所有信号波形。

基于0930（ZEBU EMU）版本，整理了关注的信号列表，算子仿真结束，抓取波形进行分析时，可以直接加载保存好的信号文件（PATH: /data/public/jiale/emu-0930.rc），具体如下：

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.clk
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe.wb_ctl_wb_en
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe.wb_ctl_wb_trid[8:0]
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe.twait_tacp_busy[1:0]
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe_tdte_iss_head
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe_tdte_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe_tdte_fin_itag_v

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst0_axi_arvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst0_axi_rvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst1_axi_arvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst1_axi_rvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst2_axi_arvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst2_axi_rvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst3_axi_arvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst3_axi_rvalid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.rmt_p_rd_ots_cnt[3:0]
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tdte_idle
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tacp_cnt[1:0]
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_ld_busy
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_st_busy
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_fin_stag_v
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_fin_ltag_v
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tlsu_idle_unused
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tlsu_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.talu_active
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.talu_tfe_tdv
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.cmp_active
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.cvt_active
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.fma_active

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.tfe_talu_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_talu.tpe_active
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_talu_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tsfu_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tsfu_tfe_td_v
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_executing
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_ts_last
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_wait_src
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_wr_bkpressur e
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_iss_valid
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_tfe_iss_ready
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_idle
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_td_id[7:0]
hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_td_rdy

基于0815（ZEBU EMU）版本，整理了关注的信号列表，如下（**仅备份**）：

所属单元	信号名称
PE	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_pe_xbar.clk
TFE	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe. wb_ctl_wb_en
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe. wb_ctl_wb_trid[8:0]
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tc_tfe. twait_tacp_busy[1:0]
DTE	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe _tdte_iss_head

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe
_tdte_iss_valid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tfe
_tdte_fin_itag_v

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st0_mif_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st1_mif_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st2_mif_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st3_mif_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st0_l2b_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st1_l2b_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st2_l2b_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st3_l2b_axi_arvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st0_mif_axi_rvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st1_mif_axi_rvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st2_mif_axi_rvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st3_mif_axi_rvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st0_l2b_axi_rvalid

hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.m
st1_l2b_axi_rvalid

	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst2_l2b_axi_rvalid
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.mst3_l2b_axi_rvalid
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.rmt_p_rd_ots_cnt[3:0]
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tdte_top.tdte_idle
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tacp_cnt[1:0]
LSU	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_ld_busy
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_st_busy
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tlsu_idle
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tlsu_tfe_fin_ltag_v
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tlsu_iss_valid
TALU	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.talu_active
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_talu_idle
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.talu_tfe_td_v
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.talu_tfe_iss_ready
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.talu_tfe_fin_valid
TSFU	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.pme_tsfu_active

	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.pme_tsfu_bf16_exec
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.pme_tsfu_fp16_exec
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.pme_tsfu_fp32_exec
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tsfu_tfe_td_v
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tsfu_iss_valid
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tsfu_tfe_fin_valid

TMAC	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_executing
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_ts_last
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_wait_src
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_wr_bkpressure
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_iss_valid
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_tfe_iss_ready
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_tfe_fin_valid
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_td_id[7:0]
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tfe_tmac_td_rdy
	hw_top.u_sipu_v1p5_top.pec_top_wrapper_inst0.u_pewa_top_0.u_pe_top.u_tcore.tmac_tfe_td[7:0]

9.4 Profiler工具-PMU



也可以参考该独立文档：[EMU环境PMU工具使用](#)

9.4.1 概述

PMU (Performance Monitoring Unit) 是 SIPU 芯片内置的硬件性能计数器模块。每个硬件模块（PE、DRAMIF、C2C 等）都有一组计数器，可以在程序运行期间记录各种事件的发生次数或周期数，具体可参考架构设计文档：[SIPU 1.0 PFM\(Performance Monitor Unit\) Subsystem架构设计方案_v1.0](#)

PMU 计数器只做累加操作，硬件上没有为中间状态值预留存储空间，因此不支持分时采样（time-sharing sampling）。软件对计数器只能执行三种操作：

1. **Reset** — 清零计数器
2. **Stop** — 停止计数
3. **Read** — 读取当前累计值

这意味着每次采集只能获得从 reset 到 stop 之间的累计总数，无法获取过程中的时间序列数据。

9.4.2 代码仓

主仓库：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool

版本发布地址：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/releases

说明文档：https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/blob/dev/README.md?ref_type=heads

技术细节介绍：

https://gitlabsoft.siorigin.com/algo/framework/pmu_tool/-/blob/dev/docs/pmu_tool_guide.md?ref_type=heads

9.4.3 基础使用方法



1. 为保证性能分析准确，请尽量使用带有buffer die的EMU环境

2. 需要根据实际运行的EMU环境，与kernel实际使用的PE，选择合适的mask profile进行指令执行

(1) 环境准备

PMU工具一般在真实设备上（或真实仿真环境，如ZEBU EMU）使用，如果在ZEBU上使用，需要首先配置启动EMU等环境，以下均以ZEBU EMU环境为例进行使用说明

(2) 版本获取

获取最新稳定版本的PMU发布版，形如：PMU Tool v20260325，可从上面列出的版本发布地址中获取。介质形式为压缩包，需要拷贝至EMU/QEMU环境中并解压

(3) 介质验证

确保QEMU已正常启动，且SIPU driver已加载，可尝试执行`./pmu_tool --help`观察是否可以正常输出内容

(4) kernel准备

准备待测试分析的kernel binary，如`tile_sum`，确保binary具有可执行权限，且在当前路径下可正常执行

(5) 性能分析指令

使用指令：`./PMU_PATH/pmu_tool ./KERNEL_PATH/tile_sum`（注意，kernel binary前需要`./`前缀）

观察仿真输出结果是否正常，因为默认pmu工具会多次执行kernel来获取完整数据，kernel输出内容也会重复多次（确保当前kernel输出结果，与单独运行kernel时，一致）

如果需要使用其他更为细致的分析方式，可参考PMU工具的用法介绍文档。

(6) 结果查看

在当前目录下，会生成output目录，内部按时间戳生成已执行pmu工具分析的kernel执行结果，形如：`output/pmu_20260326_110031/`，包含文件一般有：

代码块

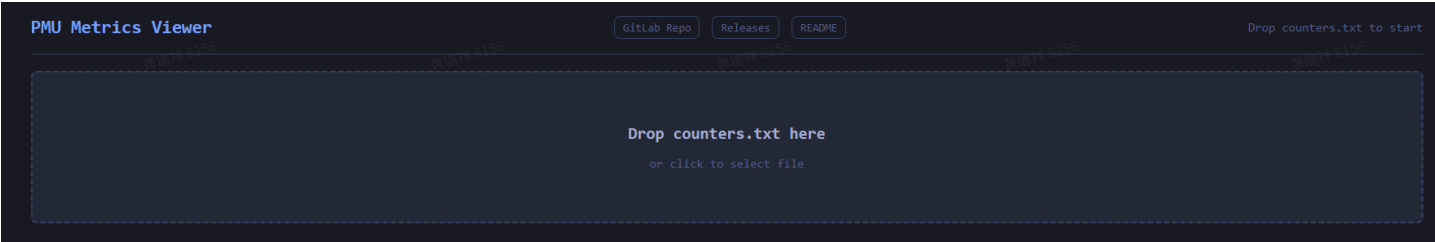
```
1  -rw-rw-r-- 1 ceshi ceshi 30295  3月 26 11:00 counters_organized.txt
2  -rw-rw-r-- 1 ceshi ceshi 45869  3月 26 11:00 counters.txt
3  -rw-rw-r-- 1 ceshi ceshi  1063  3月 26 11:00 hwconfig.txt
4  -rw-rw-r-- 1 ceshi ceshi 98669  3月 26 11:00 metrics_organized.txt
5  -rw-rw-r-- 1 ceshi ceshi 65556  3月 26 11:00 metrics.txt
6  -rw-rw-r-- 1 ceshi ceshi  3373  3月 26 11:00 rvcore_organized.txt
7  -rw-rw-r-- 1 ceshi ceshi  4385  3月 26 11:00 rvcore.txt
```

可主要关注其中的counters.txt文件，内部包含各个类型counter的信息，形如：

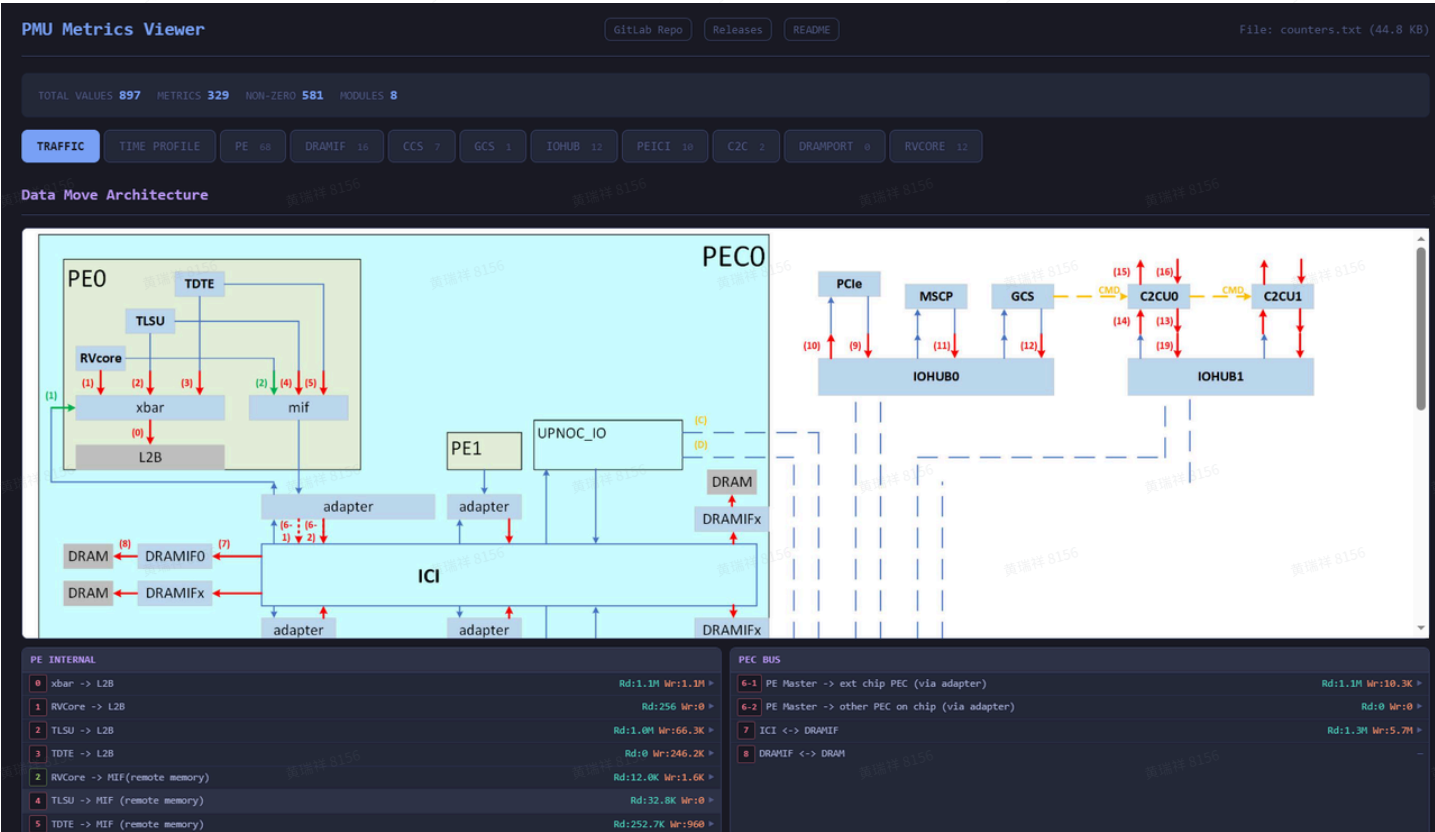
1	Device: usipu0, Module: PE, PFM: 0, Group: 0						
2	usipu0	PE	0	0	0	pe_active_cycles	0
3	usipu0	PE	0	0	1	rvcore0_active_cycles	0
4	usipu0	PE	0	0	2	rvcore1_active_cycles	0
5	usipu0	PE	0	0	3	tlsu_ld_active_cycle	0
6	usipu0	PE	0	0	4	tlsu_st_active_cycle	0
7	usipu0	PE	0	0	5	talu_active_cycles	0
8	usipu0	PE	0	0	6	tsfu_active_cycles	0
9	usipu0	PE	0	0	7	tdte_active_cycles	0
10	usipu0	PE	0	0	8	l2b_active_cycles	0
11	usipu0	PE	0	0	9	tmac_executing	0
12	usipu0	PE	0	0	10	pme_tsfu_fp16_inst_num	0
13	usipu0	PE	0	0	11	pme_tsfu_bf16_inst_num	0
14	usipu0	PE	0	0	12	pme_tsfu_fp32_inst_num	0

(7) 可视化工具

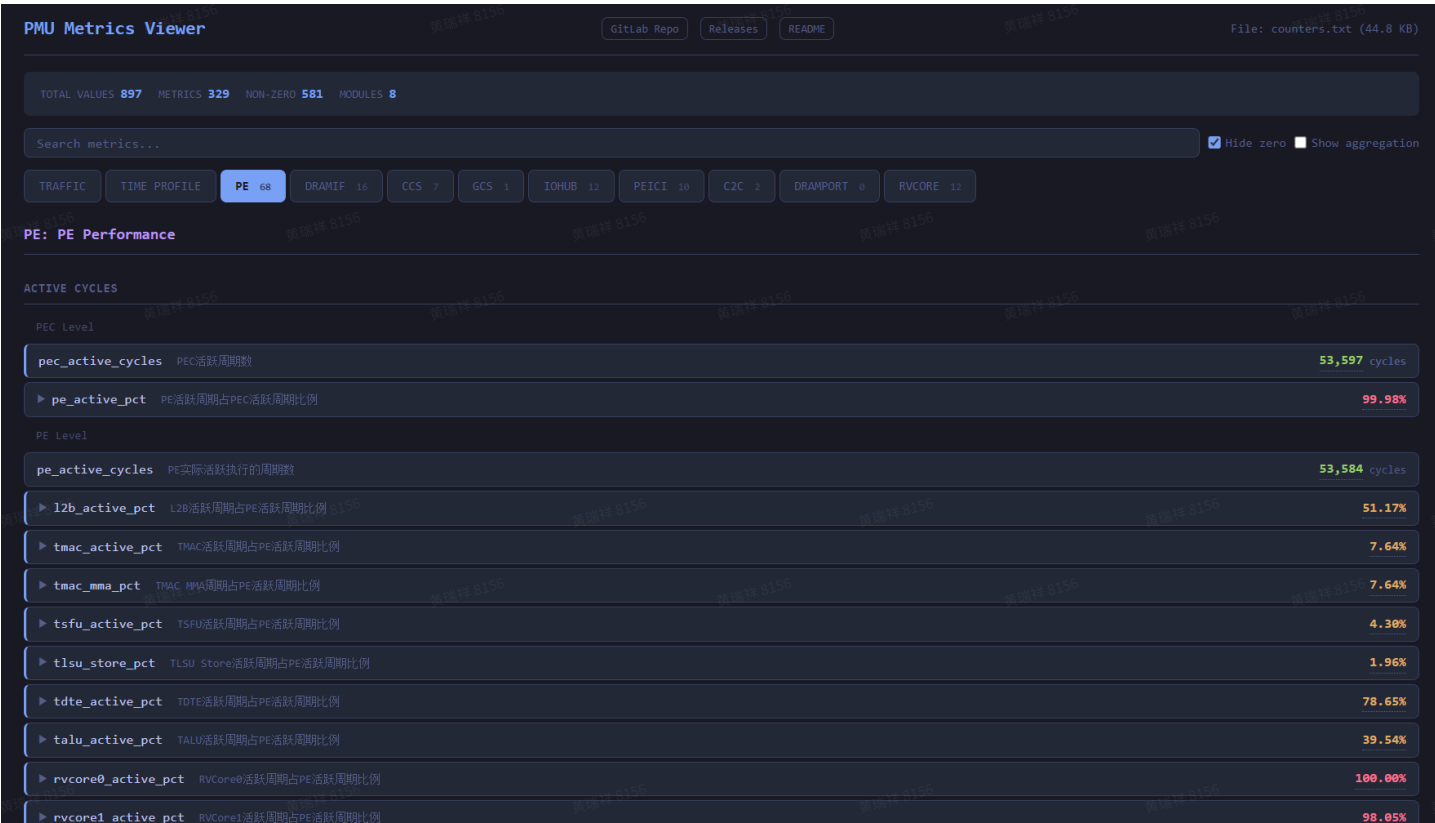
可使用可视化工具，查看counters.txt的结果，可视化工具与版本发布包在同一目录下，名为：metrics_viewer.html，可直接用浏览器打开该文件，有类似如下结果：



可用文件拖动的方式，将关注的counters.txt文件拖拽至其中，有类似如下输出：



点击PE栏目，可看到PE相关指令的执行统计情况：



也可点击其他栏目，查看关注的相关信息。

（8）AI辅助分析

也可考虑将生成的counters.txt文件投喂给AI coding工具，用于辅助分析kernel的性能和优化方向。

9.4.4 可选的profile配置

需要注意的是，当前的EMU环境分为多个版本，不同版本能够抓取的信号，也是不同的，常用的EMU版本（仅针对sikernel的运行需要）包括：

（1）S1G1C1E

最小形态，仅包含1个PE

（2）S1G1C4E

包含1个完整的PEC（内含4个PE）

（3）S1G4C4E

包含1个完整的PEG（内含4个完整的PEC）

另外还有更大的S2G4C4E，以及多卡版本，在sikernel的运行过程中，是不需要额外的这些硬件资源的，这里就不进行展开介绍了。

下面就针对上面这3种EMU版本，说明其profile的配置方法（是否带有buffer die，profile均相同；同时，sikernel不关注C2C，因此这里均不配置抓取）。

以下配置内容，需要补充到pmu工具配置文件（config/hardware_config.json）的尾部，为避免数值交叉，此处选取较大的profile id。

注意：

1. 非所有PEC或PE生效的情况下，需要结合实际EMU版本，查看active的PE或PEC的位置，可能与下面的配置有差异，比如S1G1C1E版本，可能最后1个PE为active，那么就需要把PE_MASK_PER_PEC改为[0, 0, 0, 1]

(1) S1G1C1E

代码块

```
1      "51": {
2          "_info": "S1G1C1E",
3          "PEC_MASK": [1, 0, 0, 0, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 0, 0, 0],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0],
8          "GCS_MASK": [1],
9          "IOHUB_MASK": [1, 1],
10         "IOPD_MASK": [1, 1],
11         "C2C_MASK": [0, 0]
12     }
```

(2) S1G1C4E

代码块

```
1      "52": {
2          "_info": "S1G1C4E",
3          "PEC_MASK": [1, 0, 0, 0, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 1, 1, 1],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1],
8          "GCS_MASK": [1],
9          "IOHUB_MASK": [1, 1],
10         "IOPD_MASK": [1, 1],
11         "C2C_MASK": [0, 0]
12     }
```

(3) S1G4C4E

代码块

```
1      "53": {
2          "_info": "S1G4C4E",
3          "PEC_MASK": [1, 1, 1, 1, 0, 0, 0, 0],
4          "PE_MASK_PER_PEC": [1, 1, 1, 1],
5          "CCS_MASK_PER_PEC": [1],
6          "DRAMIF_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7          "DRAMPORT_MASK_PER_PEC": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1],
8          "GCS_MASK": [1],
9          "IOHUB_MASK": [1, 1],
10         "IOPD_MASK": [1, 1],
11         "C2C_MASK": [0, 0]
12     }
```

根据实际的EMU版本，选择profile，执行PMU工具指令，如下：

代码块

```
1  # 选择S1G1C4E环境
2  ./PMU_PATH/pmu_tool --mask-profile 52 ./KERNEL_PATH/tile_sum
```

9.5 kernel执行性能统计

除了以上对kernel内部指令执行逻辑与性能的分析，通常需要统计分析算子总的执行耗时情况，有多种方法可以实现，下面分别进行介绍。



注意：统计kernel执行耗时，通常需要warmup，即先运行kernel少量次数（1-10次）来排除冷启动影响；然后在测量阶段，运行若干次（10-10000，结合kernel单次耗时情况进行选择）求取平均值（也可观察记录其中的最小值，最大值）来评估其性能。

9.5.1 端到端耗时

可以使用C++提供的标准库中的接口，来衡量kernel执行的端到端时间。此种方法的粒度最为粗糙，但是与实际使用中的情形最为接近，用法如下：

代码块

```
1  sipuStreamSynchronize(nullptr);
```



```

2  auto t0 = std::chrono::high_resolution_clock::now();
3
4  // kernel执行, 形如:
5  // kernel<<<grid, block>>>(...);

                                sipuStreamSynchronize(nullptr);

                                auto t1 =
std::chrono::high_resolution_clock::now();
6
7  std::cout << "kernel time: " << duration.count() << " us" << std::endl;

```

9.5.2 EVENT接口

EVENT接口是SIPU runtime提供的轻量化时间统计接口, 用于精确指定要测量的代码段, 不需要额外链接库。EVENT接口的使用, 不限于单个kernel, 可测量多个kernel、memcpy的组合耗时。

需要注意的是, EVENT接口命令自身在stream中排队存在微小延迟(3000+左右cycle数), 所以统计结果相比单纯的kernel执行时间会稍大一些。但在kernel耗时 >100μs 的情况下, 差异可忽略。

使用方法如下:

代码块

```

1  #define SIKERNEL_PERF_EVENT_DECL(tag) \
2      sipuEvent_t __sikernel_perf_evt_start_##tag, \
3          __sikernel_perf_evt_stop_##tag; \
4      float __sikernel_perf_elapsed_ms_##tag = 0.0f; \
5      sipuEventCreate(&__sikernel_perf_evt_start_##tag); \
6      sipuEventCreate(&__sikernel_perf_evt_stop_##tag) \
7
8  #define SIKERNEL_PERF_EVENT_START(tag) \
9      sipuEventRecord(__sikernel_perf_evt_start_##tag) \
10
11 #define SIKERNEL_PERF_EVENT_STOP(tag) \
12     do { \
13         sipuEventRecord(__sikernel_perf_evt_stop_##tag); \
14         sipuEventSynchronize(__sikernel_perf_evt_stop_##tag); \
15         sipuEventElapsedTime(&__sikernel_perf_elapsed_ms_##tag, \
16             __sikernel_perf_evt_start_##tag, \
17             __sikernel_perf_evt_stop_##tag); \
18     } while (0) \
19
20 #define SIKERNEL_PERF_EVENT_ELAPSED(tag) \
21     (__sikernel_perf_elapsed_ms_##tag) \
22

```

```

23 #define SIKERNEL_PERF_EVENT_PRINT(tag, name) \
24     printf("[PERF] %-40s : %.3f ms\n", name, \
25           __skernel_perf_elapsed_ms_##tag) \
26 \
27 #define SIKERNEL_PERF_EVENT_DESTROY(tag) \
28     do { \
29         sipuEventDestroy(__skernel_perf_evt_start_##tag); \
30         sipuEventDestroy(__skernel_perf_evt_stop_##tag); \
31     } while (0) \
32 \
33 // kernel为输入参数，代表kernel的名称TAG，可作为区分不同kernel的依据
34 SIKERNEL_PERF_EVENT_DECL(kernel);
35 SIKERNEL_PERF_EVENT_START(kernel);
36 \
37 // kernel执行，形如：
38 // kernel<<<grid, block>>>(...);
39 \
40 SIKERNEL_PERF_EVENT_STOP(kernel);
41 SIKERNEL_PERF_EVENT_PRINT(kernel, "kernel");
42 SIKERNEL_PERF_EVENT_DESTROY(kernel);

```

9.5.3 [TODO]PTI接口

9.5.4 PMU工具

9.4中介绍的PMU工具，也可以用来获取kernel的执行性能情况。通过查询counters.txt中的相关字段（pe_active_cycles），即可获取kernel的执行时间（cycle数）：

代码块

```

1 # Device: usipu0, Module: PE, PFM: 0, Group: 0
2 usipu0 PE 0 0 0 pe_active_cycles 53584
3 usipu0 PE 0 0 1 rvcore0_active_cycles 53583
4 usipu0 PE 0 0 2 rvcore1_active_cycles 52539
5 usipu0 PE 0 0 3 tlsu_ld_active_cycle 0
6 usipu0 PE 0 0 4 tlsu_st_active_cycle 1050
7 usipu0 PE 0 0 5 talu_active_cycles 21189
8 usipu0 PE 0 0 6 tsfu_active_cycles 2808
9 usipu0 PE 0 0 7 tdte_active_cycles 42143
10 usipu0 PE 0 0 8 l2b_active_cycles 27421

```

